2(a)	One mark per point: 1 Open the file in append mode (and subsequently close) **before loop** 2 Initialise a variable to 1 / 0 **before loop** 3 Use the variable as an index to the array to access an element **in a loop** 4 Test if element value is blank / empty **in a loop** 5 If not blank then write the value to the file **in a loop** 6 Increment the variable **in a loop** 7 Repeat from MP3 until variable value is 66 / 65 / all elements (in the array) have been checked **Max 6**	**6**

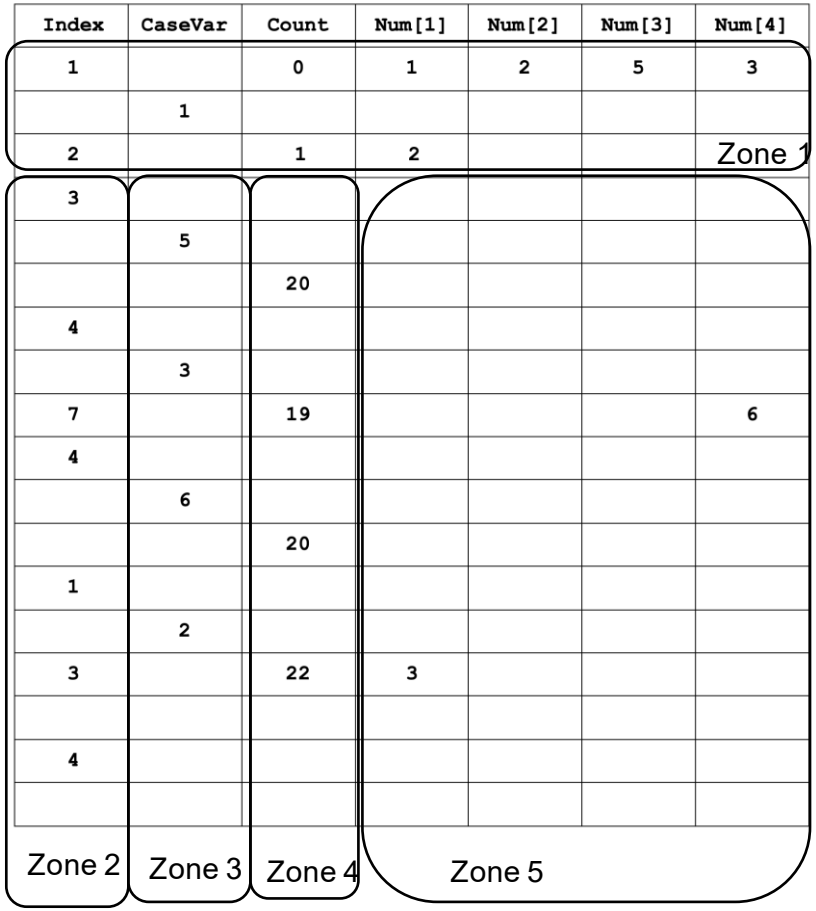
Examiner report missing
9618_w25_qp_23 Question 2

3	FUNCTION Pop() RETURNS PopData DECLARE ThisPop : **PopData** MP 2 IF **SP < 1 // SP = 0** THEN MP 3 PopData.Exists ← **FALSE** // Stack is empty MP 1 ELSE PopData.Data ← **ThisStack[SP]** MP 4 PopData.Exists ← **TRUE** MP 1 SP ← **SP – 1** MP 5 ENDIF RETURN ThisPop` ENDFUNCTION Mark as follows: MP 1 One mark for **both** assignments to PopData.Exists (FALSE **and** TRUE) MP 2, 3, 4 ,5 One mark for each of remaining four bold parts	**5**

Examiner report missing
9618_w25_qp_23 Question 3

4	Example solution:
	
	`PROCEDURE Store()`
	`  DECLARE Index, StartPos, ThisNum : INTEGER`
	`  DECLARE ThisString : STRING`
	
	`  Index ← 1`
	`  StartPos ← 1`
	
	`  WHILE StartPos < LENGTH(NumString) AND Index < 21`
	`  ThisString ← MID(NumString, StartPos, 3)`
	`  ThisNum ← STR_TO_NUM(ThisString)`
	`  Data[Index] ← ThisNum`
	`  Index ← Index + 1`
	`  StartPos ← StartPos + 4`
	`  ENDWHILE`
	
	`ENDPROCEDURE`
	
	Mark as follows:
	1 Declare all local variables used
	2 Conditional loop while end of string not reached
	3 ... and array not full
	4 Attempted use of `MID()` // Attempted use of `Right()` and `Left()`
	5 Correct extraction and use next three letter substring from `NumString` **in a loop**
	6 Convert to integer **and** assign value to array element **in a loop**
	7 Correct increment of array index by 1 **and** start position of substring by 4
	
	Max 6
	
	Alternative FOR loop solution:
	
	`PROCEDURE Store()`
	`  DECLARE Index, StartPos, ThisNum : INTEGER`
	`  DECLARE ThisString : STRING`
	`  StartPos ← 1`
	`  FOR Index ← 1 TO 20`
	`  ThisString ← MID(NumString, StartPos, 3)`
	`  ThisNum ← STR_TO_NUM(ThisString)`
	`  Data[Index] ← ThisNum`
	`  StartPos ← StartPos + 4`
	`  IF StartPos > LENGTH(NumString) THEN`
	`  Break`
	`  ENDIF`
	`  NEXT Index`
	`ENDPROCEDURE`
		6

Examiner report missing
9618_w25_qp_23 Question 4

5(a)	 Zones 1 to 4 : One mark for each set of values as shown Zone 5: Num[4] set to 6 when Index is 7 **and** Num[1] set to 3 when index is 3 for **second** time	**5**

Examiner report missing
9618_w25_qp_23 Question 5

8(a)	Example solution:
	
	`PROCEDURE LoanStatus(ThisStudentID, ThisBookID : STRING)`
	`  DECLARE Index : INTEGER`
	`  DECLARE Found : BOOLEAN`
	
	`  Index ← 1 / 0`
	`  Found ← FALSE`
	
	`  WHILE Index <= 8000 / 7999 AND NOT Found`
	`  IF Loan[Index].StudentID = ThisStudentID AND`
	`  Loan[Index].BookID = ThisBookID THEN`
	`  IF Loan[Index].OnLoan = FALSE THEN`
	`  OUTPUT "Loan has been returned."`
	`  ELSE`
	`  OUTPUT "Loan has not been returned."`
	`  ENDIF`
	`  Found ← TRUE`
	`  ENDIF`
	`  Index ← Index + 1`
	`  ENDWHILE`
	
	`  IF NOT Found THEN`
	`  OUTPUT "Warning - Loan not found."`
	`  ENDIF`
	`ENDPROCEDURE`
	
	Mark as follows:
	1 Procedure heading, parameters and ending
	2 Loop through **all elements** in `Loan` array
	3 ... or until record loan is found
	4 Test for correct `StudentID` **in a loop**
	5 ... and correct `BookID`
	6 ... check `OnLoan` data item
	7 ... and output <u>both</u> messages as appropriate regarding book loan status **once**
	8 If no matching loan found, Output 'not found' **after the loop**
		8

8(b)	Example solution:
	
	`FUNCTION LoansPerTutor(TutorID : STRING) RETURNS INTEGER`
	`  DECLARE Index, Count : INTEGER`
	
	`  Count ← 0`
	
	`  FOR Index ← 1 / 0 TO 8000 / 7999`
	`  IF LEFT(Loan[Index].StudentID, 3) = TutorID AND`
	`  Loan[Index].OnLoan = TRUE THEN`
	`  Count ← Count + 1`
	`  ENDIF`
	`  NEXT Index`
	
	`  RETURN Count`
	
	`ENDFUNCTION`
	
	Mark as follows:
	1 Initialise local integer for `Count`
	2 Loop through **all elements** in the `Loan` array
	3 Attempt at referencing a data item **in a loop**
	4 Extract `TutorID` from `Loan[Index].StudentID` **in a loop**
	5 Test for Matching Tutor **AND**
	test if the book has been returned using correct dot notation **in a loop**
	6 ... if true then increment `Count` **in a loop**
	7 Return count
		7

8(c)(i)	One mark per point: 1 Include (a suffix string based on) the date of the archive / the next number in sequence 2 Concatenate with a root filename such as 'Archive'	**2**
8(c)(ii)	Boolean data cannot be written to a text file // `OnLoan` is not a string // `OnLoan` will have to be converted to text	**1**

8(c)(iii)	One mark per point: Benefit: Algorithm to **store / extract** a record is easier // unpacking of data is not required // No need for string concatenation Drawback: More file **accesses needed** (to read / write a complete record)	**2**

Examiner report missing
9618_w25_qp_23 Question 8

1(a)

Pseudocode example	Selection	Iteration	Subroutine

`IF Status = FALSE THEN`
`  FOR Count ← 1 TO 20`
`  CALL Reset(Count)`
`  NEXT Count`
`ENDIF`
	✓	✓	✓

`OTHERWISE : Status ← TRUE`	✓		
`WHILE AllDone() = TRUE`		✓	✓
`30 : NextChar ← 'X'`	✓		

One mark per row

4

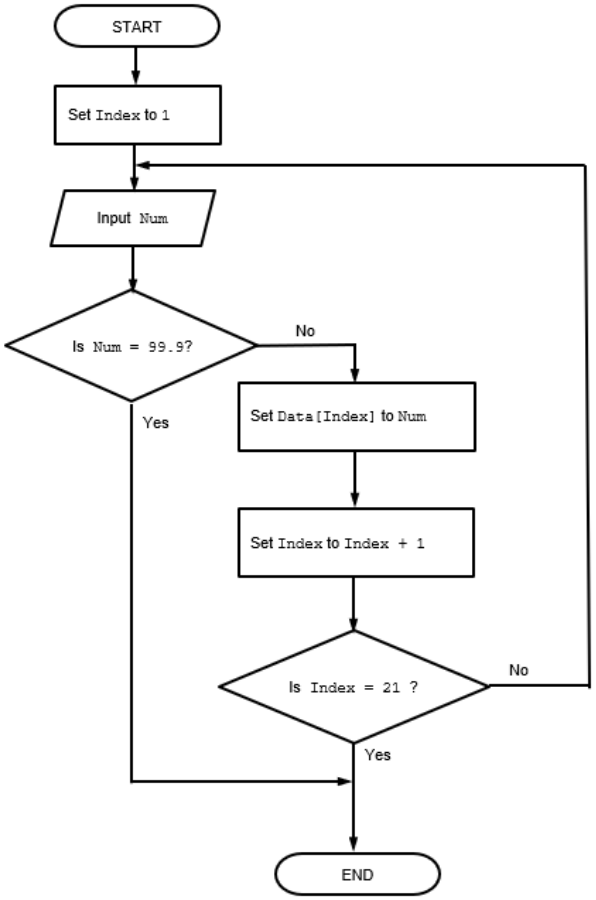
1(b)

Variable	Example data value	Data type
`Result`	`5.42`	**REAL**
`MonthLetter`	`"JFMAMJJASOND"`	**STRING**
`Birthday`	`15/11/2009`	**DATE**

One mark per row

3

Examiner report missing
9618_w25_qp_22 Question 1

2	 Mark as follows: **MP1** Initialisation of the array index **MP2** Input number **in a loop** **MP3** Test for termination value (99.9) **and** correct ending **MP4** Test for array full / maximum 20 iterations **and** correct ending **MP5** Assign number to `Data` array element **and** increment index	**5**

Examiner report missing
9618_w25_qp_22 Question 2

3(a)	Mark as follows:		**6**

MP1	1		open the file **OldFile.txt** in **read (mode)**
MP2	2		open the file **NewFile.txt** in **write (mode)**
	3		read a line from **OldFile.txt** and store in **`LineX`**
MP3	4		read a line from **OldFile.txt** and store in **`LineY`**
	5		read a line from **OldFile.txt** and store in **`LineZ`**
MP4	6		set `NewString` to **`LineX & '\' & LineY & '\' & LineZ`**
MP5	7		write `NewString` to **NewFile.txt**
MP6	8		repeat from step **3** until **end of OldFile.txt**
	9		close both files

3(b)	It does not appear in (any of) the data items // it does not appear in the ID **and** Name	**1**

3(c)	**MP1** Add a (**single)** new separator **at the end of the third (original) item** **MP2** Calculate the **length** of **both new data items** **MP3** Store / append the **length** (of the new each items) **MP4** as a fixed **length** / separated digit string // by example **Max 3**	**3**

Examiner report missing
9618_w25_qp_22 Question 3

4(a)	Example solution:
	
	`DECLARE CheckTotal : ARRAY [0:35] OF BOOLEAN`
	`DECLARE Index : INTEGER`
	
	`FOR Index ← 0 TO 35`
	`  CheckTotal[Index] ← FALSE`
	`NEXT Index`
	
	Mark as follows:
	
	MP1 Declaration of `CheckTotal` array
	MP2 Declaration of a loop counter
	MP3 Loop – with correct syntax and number of iterations
	MP4 Assignment of array element to `FALSE`
		4

4(b)	`DECLARE EasyQ, HardQ : INTEGER`
	
	`FOR EasyQ ← 0 TO 15 STEP 3`
	MP1 MP2
	
	`  FOR HardQ ← 0 TO 20 STEP 5`
	MP3 MP4
	
	`  CheckTotal[HardQ + EasyQ] ← TRUE`
	MP5
	
	`  NEXT HardQ`
	
	`NEXT EasyQ`
	
	Mark as follows:
	One mark per gap (boldened)
		5

Examiner report missing
9618_w25_qp_22 Question 4

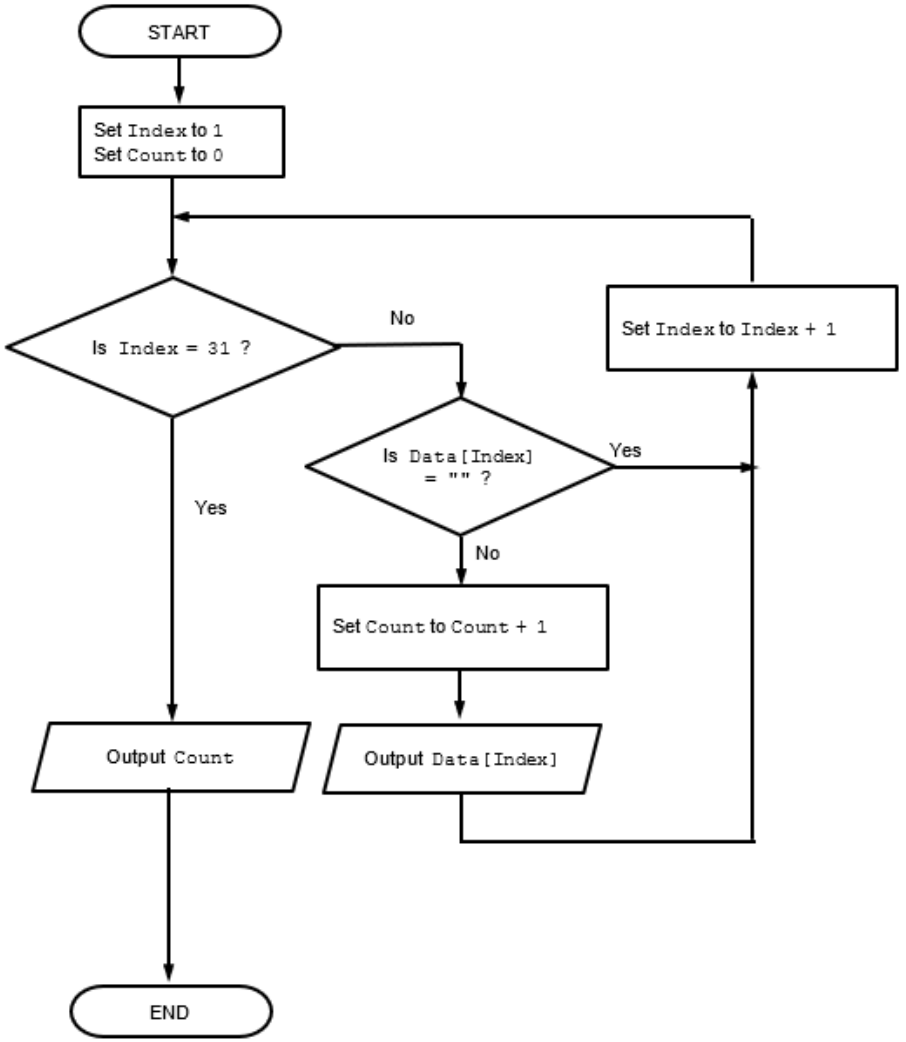
8(a)	Example solution:
	
	`PROCEDURE CountLoans(ThisStudentID : STRING)`
	`  DECLARE Index, LCount, RCount : INTEGER`
	
	`  LCount ← 0`
	`  RCount ← 0`
	
	`  FOR Index ← 1 TO 7000`
	`  IF Loan[Index].StudentID = ThisStudentID THEN`
	`  IF Loan[Index].OnLoan = FALSE THEN`
	`  RCount ← RCount + 1`
	`  ELSE`
	`  LCount ← LCount + 1`
	`  ENDIF`
	`  ENDIF`
	`  NEXT Index`
	
	`  OUTPUT "Student has ", LCount, " books on loan"`
	`  OUTPUT "and has returned ", RCount, " books"`
	
	`ENDPROCEDURE`
	
	Mark as follows:
	MP1 Declaration of Index **and** two count variables
	MP2 Loop for 7000 iterations
	MP3 `Index` references an **indexed data** item with correct **.dot notation**
	MP4 Attempt to compare the `StudentID` field = **parameter in a loop**
	MP5 Test of the `OnLoan` field **and** correct logic for both outcomes
	MP6 Both count variables initialised **and** increment appropriate count **in a loop**
	MP7 Output includes both counts with a <u>suitable</u> message
		7

8(b)	Example solution:
	
	`FUNCTION NewLoan(ThisStudentID, ThisBookID : STRING) __ RETURNS BOOLEAN`
	`  DECLARE Index : INTEGER`
	`  DECLARE IsStored : BOOLEAN`
	`  CONSTANT Blank = ""`
	
	`  Index ← 1`
	`  IsStored ← FALSE`
	
	`  WHILE Index <= 7000 AND NOT IsStored`
	`  IF Loan[Index].StudentID = Blank THEN`
	`  Loan[Index].StudentID ← ThisStudentID`
	`  Loan[Index].BookID ← ThisBookID`
	`  Loan[Index].OnLoan ← TRUE`
	`  IsStored ← TRUE`
	`  ENDIF`
	`  Index ← Index + 1`
	`  ENDWHILE`
	
	`  RETURN IsStored`
	`ENDFUNCTION`
	
	Mark as follows:
	
	MP1 Loop through 7000 elements in `Loan` array – correct loop structure
	MP2 or until the first unused element is found // Correct `RETURN` inside a `FOR` loop
	MP3 Correct use of the indexed dot notation
	MP4 Test if record is unused (`.StudentID = ""`) **in a loop**
	MP5 Two item assignment statements correct
	MP6 `Loan[Index].OnLoan` assigned TRUE
	MP7 Correct logic to return `BOOLEAN` in both cases (position found / no position available)
	
	Alternative solution: loop only (no 'found' mechanism)
	
	`  FOR Index ← 1 TO 7000`
	`  IF Loan[Index].StudentID = Blank THEN`
	`  Loan[Index].StudentID ← ThisStudentID`
	`  Loan[Index].BookID ← ThisBookID`
	`  Loan[Index].OnLoan ← TRUE`
	`  RETURN TRUE`
	`  ENDIF`
	`  NEXT`
	`  RETURN FALSE`
		7

8(c)	**MP1** Save to/Open a **(text) file** **MP2** **Single line** – form a string with separator characters and write to the file // **Multiple lines** - write each data item to a separate line in the file	**2**

8(d)	Additional date(s) solution **MP1** Store the **return date** of each loan // the **start date <u>and</u>** the **loan period** **MP2** Algorithm will do a **calculation** involving the MP1 data item(s) OR The student email address solution **MP3** Store the student <u>email address</u> **MP4** In order **to send** the email // the student is alerted to return the book **Max 2**	**2**

Examiner report missing
9618_w25_qp_22 Question 8

2	Example solution  Mark as follows: 1 Initialisation of variables used as **array index** and **count** 2 Loop through exactly 30 elements in the array 3 Test for empty string in current array element // Test for non-empty string 4 Increment count if appropriate **in a loop** 5 Both required outputs under correct conditions with correct outputs	**5**

Examiner report missing
9618_w25_qp_21 Question 2

3(a)(i)	Pseudocode:
	
	`TYPE RentalRecord`
	`  DECLARE RentalID : STRING`
	`  DECLARE CarID : INTEGER`
	`  DECLARE DisCode : CHAR`
	`  DECLARE Start : DATE`
	`  DECLARE Duration : INTEGER`
	`  DECLARE Completed : BOOLEAN`
	`ENDTYPE`
	
	Mark as follows:
	
	1 One mark for `TYPE RentalRecord` and `ENDTYPE` statements
	2 One mark for `RentalID` and `CarID` declared correctly
	3 One mark for `DisCode` and `Start` declared correctly
	4 One mark for `Duration` and `Completed` declared correctly
		4

3(a)(ii)	`DECLARE Rental : ARRAY[1:500] OF RentalRecord` One mark per underlined phrase	**2**
3(b)	One mark per point: 1 **Different data types** held (for each rental) under a single identifier 2 Allows for iteration through rentals/records // Can access **rentals/records** in a loop // Can (directly) access a specific rental/record using an (array) index 3 Simplifies the code/program / less code needed / reduces code duplication // Makes **code/program** easier to understand // Makes testing / debugging easier // **Program** maintenance easier	**3**

Examiner report missing
9618_w25_qp_21 Question 3

4(a)	Example solution - conditional:
	
	`PROCEDURE Store()`
	`  DECLARE ThisNum, LastNum : REAL`
	`  DECLARE Index : INTEGER // index to global array`
	
	`  INPUT ThisNum`
	`  LastNum ← ThisNum`
	`  Number[1] ← ThisNum`
	
	`  INPUT ThisNum`
	`  Index ← 2`
	
	`  WHILE Index < 21 AND ThisNum > LastNum`
	`  Number[Index] ← ThisNum`
	`  LastNum ← ThisNum`
	`  Index ← Index + 1`
	`  IF Index < 21 THEN`
	`  INPUT ThisNum`
	`  ENDIF`
	`  ENDWHILE`
	
	`  OUTPUT Index - 1, " values were stored in the array"`
	`ENDPROCEDURE`
	
	Alternative loop:
	`  // Loop without using LastNum`
	`  WHILE Index < 21 AND ThisNumNum > Number[Index - 1]`
	`  Number[Index] ← ThisNum`
	`  Index ← Index + 1`
	`  IF Index < 21 THEN  //skip input if array full`
	`  INPUT ThisNum`
	`  ENDIF`
	`  ENDWHILE`
	
	Mark as follows:
	1 Declare local variables `Index` as integer, `ThisNum` and `LastNum` as real
	2 Input value and assign to first element
	3 **Attempt** at conditional loop including **both** tests
	4 **Completely correct** conditional loop including **both** tests
	5 Assign input value to current element if greater than previous element **and** input next value **in a loop**
	6 Final output giving attempted count of elements stored plus message **after a loop**
		6

4(b)(i)	One mark per point: 1 The amount of values **stored** (in a text file) is **not fixed** / **not limited** (other than by secondary storage available) // The amount values **stored** is **not limited** to the size of the array 2 The data is stored **permanently** / **non-volatile** // Data can be restored / reused without **re-entering** values	**2**

4(b)(ii)	Each (real) value would have to be **converted** to a string	**1**

Examiner report missing
9618_w25_qp_21 Question 4

5(a)(i)	It is a value that is **invalid as an** (array) **index** // not in the **range** of valid (index) values	**1**
5(a)(ii)	To test whether the **current element / value at index** matches the **given / search** string / data / **parameter**	**1**
5(a)(iii)	The value of `FoundAt` is only updated to the (current) index if it currently stores the initial value / -1 // `FoundAt` is not updated when it currently stores any other value than -1	**1**

Examiner report missing
9618_w25_qp_21 Question 5

8(a)	Example solution:
	
	`PROCEDURE OKToBorrow(ThisStudentID : STRING)`
	`  DECLARE Index, Count : INTEGER`
	
	`  CONSTANT Max = 5`
	`  Count ← 0`
	`  Index ← 1 / 0`
	
	`  WHILE Index <= 5000 / Index <= 4999 AND Count < Max`
	`  IF Loan[Index].StudentID = ThisStudentID AND __`
	`  Loan[Index].OnLoan =`
	`TRUE THEN`
	`  Count ← Count + 1`
	`  ENDIF`
	`  Index ← Index + 1`
	`  ENDWHILE`
	
	`  IF Count < Max THEN`
	`  OUTPUT "Student may borrow another book"`
	`  ELSE`
	`  OUTPUT "Student may not borrow another book"`
	`  ENDIF`
	
	`ENDPROCEDURE`
	
	Mark as follows:
	1 Procedure heading, parameter and ending
	2 Loop through **all elements** in `Loan` array
	3 ... terminating if `Max` reached
	
	4 Test for correct `StudentID` field **in a loop**
	5 ... and that `OnLoan = TRUE` **in a loop**
	... if so, then increment `Count` **in a loop**
	6 Output an appropriate **message** in both cases **after the loop**
		7

8(b)	Example solution:
	
	`FUNCTION ReturnBook(ThisStudentID, ThisBookID : STRING)`
	`  RETURNS BOOLEAN`
	`  DECLARE Index : INTEGER`
	
	`  Index ← 1`
	
	`  WHILE Index < 5001`
	`  IF Loan[Index].StudentID = ThisStudentID  AND __`
	`  Loan[Index].BookID = ThisBookID THEN`
	`  IF Loan[Index].OnLoan = TRUE THEN // Not returned`
	`  Loan[Index].OnLoan ← FALSE // Mark as returned now`
	`  RETURN TRUE  //Found and not already returned`
	`  ELSE`
	`  RETURN FALSE  // Found and already returned`
	`  ENDIF`
	`  ENDIF`
	`  Index ← Index + 1`
	`  ENDWHILE`
	
	`  RETURN False // loan not found`
	`ENDFUNCTION`
	
	Mark as follows:
	1 Loop through **all elements** in `Loan` array
	2 ...and book loan not found
	3 Attempt to reference an individual data item **in a loop**
	4 Correct test for `StudentID` **and** correct `BookID` **in a loop**
	5 **...** If book not returned yet, then set `OnLoan` field to `FALSE`
	6 RETURN TRUE if book loan Found and not already returned
	7 RETURN FALSE if book loan Found and loan already returned
	8 RETURN FALSE if book loan not found
	
	Alterantive example solution – not immediate return
	
	`FUNCTION ReturnBook(ThisStudentID, ThisBookID : STRING)`
	`  RETURNS BOOLEAN`
	`  DECLARE Index : INTEGER`
	`  DECLARE Found, OK : BOOLEAN`
	
	`  Index ← 1`
	`  Found ← FALSE`
	`  OK ← TRUE`
		8

8(c)	Award marks for reference to following design features: Change to existing Record: 1 Store date due back as a new data item / in the loan record 2 Store date of loan **and** the loan length category as new data items / in the loan record **Max 1** New Record 3 **New** Records with Category (and Loan length field) Change to Program: 4 (Use the loan length category of a book to) calculate the date due date 5 Compare due date with today's date (to check if overdue / to calculate fine) 6 Map loan length category to length of loan / date due back **Max 1** **Max 2 marks**	**2**

Examiner report missing
9618_w25_qp_21 Question 8

1(a)(i)	**MP1** When a **task/module is repeated** / reused / called // performed in several places **MP2** A **specific task** can be identified /coded as a module / subroutine / procedure / function. **MP3** Reduces complexity of **program / code** // **Program / code** is simplified **MP4** Module / subroutine / procedure / function **already available**. **MP5** Testing / debugging / maintenance is easier **Max 2**	**2**

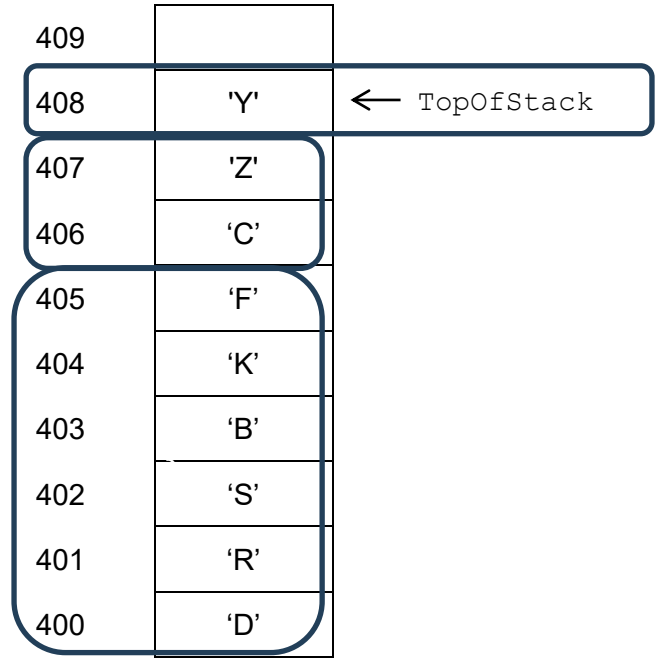
1(c)		**4**

Variable	Data type
MemberCount	**INTEGER**
TotalTakings	**REAL**
BookingConfirmed	**BOOLEAN**
MemberDOB	**DATE**

Mark as follows:
1 mark per row

Examiner report missing
9618_s25_qp_23 Question 1

2(a)(i)	**MP1** 400 **MP2** 6	**2**

2(a)(ii)	**Stack**  **MP1** TopOfStack pointing to 'Y' **and** value 'Y' in location 408 **MP2** Values 'C' and 'Z' in 406 and 407 **MP3** Values 'D' to 'F' unchanged in 400 to 405 **and** 409 is empty	**3**

2(b)(i)	**Index** **Data** **Pointer** 0 "Neptune" **2** **MP3** 1 "Jupiter" **4** 2 "Saturn" **5** 3 "Earth" **1** **MP1** 4 "Mercury" **0** 5 "Uranus" **-1** **MP2** **MP1** Earth pointer 1 **MP2** Uranus pointer -1 **MP3** Other four correct pointers	**3**
2(b)(ii)	**3**	**1**
2(c)	**MP1** First value added to queue is the first value removed // FIFO // LILO **MP2** Two **pointers** needed to indicate the **start** and **end** of the queue **MP3** May behave as a circular queue **MP4** Manipulated using `enqueue()` / `dequeue()` operations // by description **Max 2**	**2**

Examiner report missing
9618_s25_qp_23 Question 2

4(a)	**MP1** **Open** the file (`Sales.txt`) in **read mode and** subsequently close **MP2** **Read <u>a line</u>** (from the text file) **MP3** Convert the line read (string) to a number **MP4** If the number is greater than 500 **and** then output week number **MP5** Increment week number (must have been Initialised **…**) **MP6** Repeat from step 2 for 52 times // Repeat from step 2 until end of file **Max 5**	**5**

Examiner report missing
9618_s25_qp_23 Question 4

7(a)(i)	Example solution:
	
	`PROCEDURE UpdateVisit(BYREF LastVisitDate : DATE,__`
	`BYREF LoyaltyPoints : INTEGER)`
	
	`IF MONTH(TODAY()) = MONTH(LastVisitDate) THEN`
	`LoyaltyPoints ← LoyaltyPoints + 4`
	`ELSE`
	`LoyaltyPoints ← LoyaltyPoints + 1`
	`ENDIF`
	
	`LastVisitDate ← TODAY()`
	
	`ENDPROCEDURE`
	
	MP1 Procedure heading **and** ending **and** two parameters of correct type
	MP2 … both passed `BYREF`
	
	MP3 Use of `TODAY()` function to obtain current date
	MP4 Use `MONTH` function (×2) **and** one parameter is the header parameter **and** 2nd parameter is `TODAY()` / assigned variable
	
	MP5 If visit is same month, increase `LoyaltyPoints` by 4 otherwise increase by 1
	MP6 `LastVisitDate` set to `TODAY()` / 'current date' variable (assigned or unassigned)
	Max 5
		5

7(b)(i)	Example solution:
	
	`FUNCTION (MondayCheck(CustomerID : STRING) RETURNS INTEGER`
	`  DECLARE LoyaltyPoints : INTEGER`
	`  DECLARE LoyaltyPointsString : STRING`
	`  DECLARE Line : STRING`
	
	`  Line ← FindCustomer(CustomerID)`
	`  LoyaltyPointsString ← MID(Line, 8, LENGTH(Line) - 16))`
	`  LoyaltyPoints ← STR_TO_NUM(LoyaltyPointsString)`
	
	`IF DAYINDEX(TODAY()) = 2 THEN`
	`  LoyaltyPoints ← LoyaltyPoints + 10`
	`  ENDIF`
	
	`  RETURN LoyaltyPoints`
	`ENDFUNCTION`
	
	MP1 Value is returned from `FindCustomer(CustomerID)` and stored
	
	MP2 Function `MID` used
	MP3 Attempt to extract `LoyaltyPoints` from the string returned by `FindCustomer()`
	MP4 Correct extraction of `LoyaltyPoints` from the string returned by `FindCustomer()`
	
	MP5 Conversion of `LoyaltyPoints` extracted to an integer
	
	MP6 Check - is today's date a Monday using **`TODAY()`**
	MP7 … increase `LoyaltyPoints` by 10
	MP8 Return `LoyaltyPoints`
		8

7(b)(ii)	Example solution:
	
	`DayInt ← STR_TO_NUM(LEFT(LastVisitDateString, 2))`
	`MonthInt ← STR_TO_NUM(MID(LastVisitDateString, 3, 2))`
	`YearInt ← STR_TO_NUM(RIGHT(LastVisitDateString, 4))`
	
	`LastVisitDate ← SETDATE(DayInt, MonthInt, YearInt)`
	
	Mark as follows:
	
	MP1 Use of one substring function
	MP2 `DayInt`, `MonthInt` and `YearInt` all correctly extracted
	
	MP3 Use of `STR_TO_NUM` x 3
	
	MP4 `SETDATE()` function used to convert to a date type **and** assigned to `LastVisitDate`
		4

Examiner report missing
9618_s25_qp_23 Question 7

1(a)(i)	One mark for Choice of program development cycle depends on the **approach/method** required to produce the program **Or** One mark for a factor that would influence the choice of program development cycle, e.g. • Rapid development of program/prototypes required • Need for prototypes (at an early stage) • Complexity of problem • Skills / experience of development team / programmer(s) • Budget / Time / Resources available • Size of development team • Developer / programmer can return to earlier **stages** / any stage • Avoidance of repeating previous stages in development of program • How much involvement clients will have • Allows the (program) requirements to be changed during program development • (Program) requirements agreed at start of development of program **Max 1**	**1**

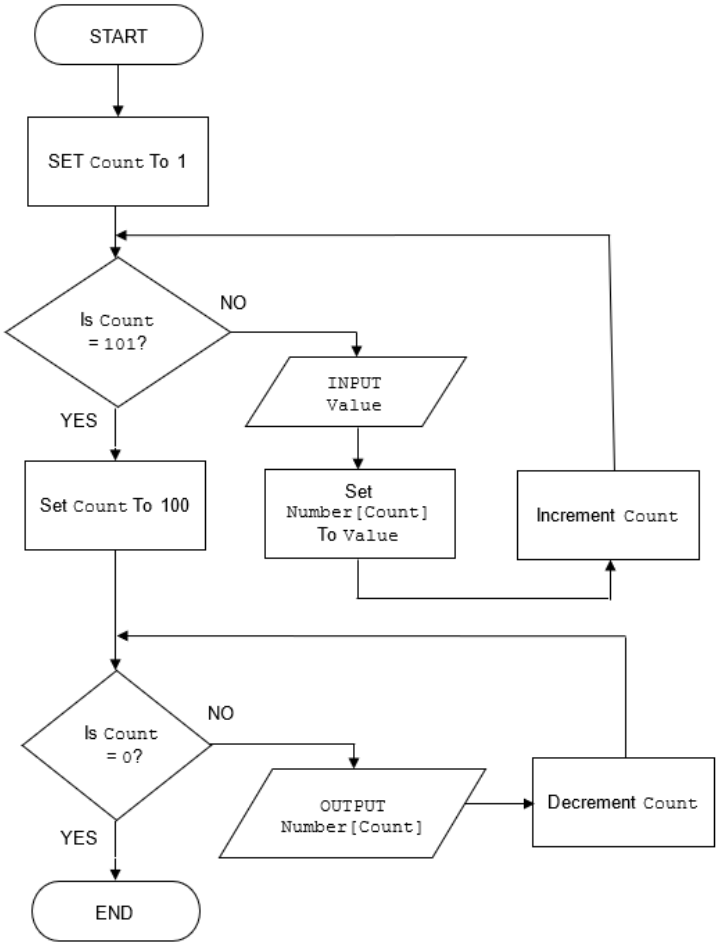
1(c)	1 mark for each correct row | Expression	Data type
---	---
`RIGHT(MachineCode, 4)`	**STRING**
`Speed * 2.5`	**REAL**
`NOT Status`	**BOOLEAN**
`IS_NUM(Check)`	**BOOLEAN**		**4**

1(d)(i)	Two / 2	**1**

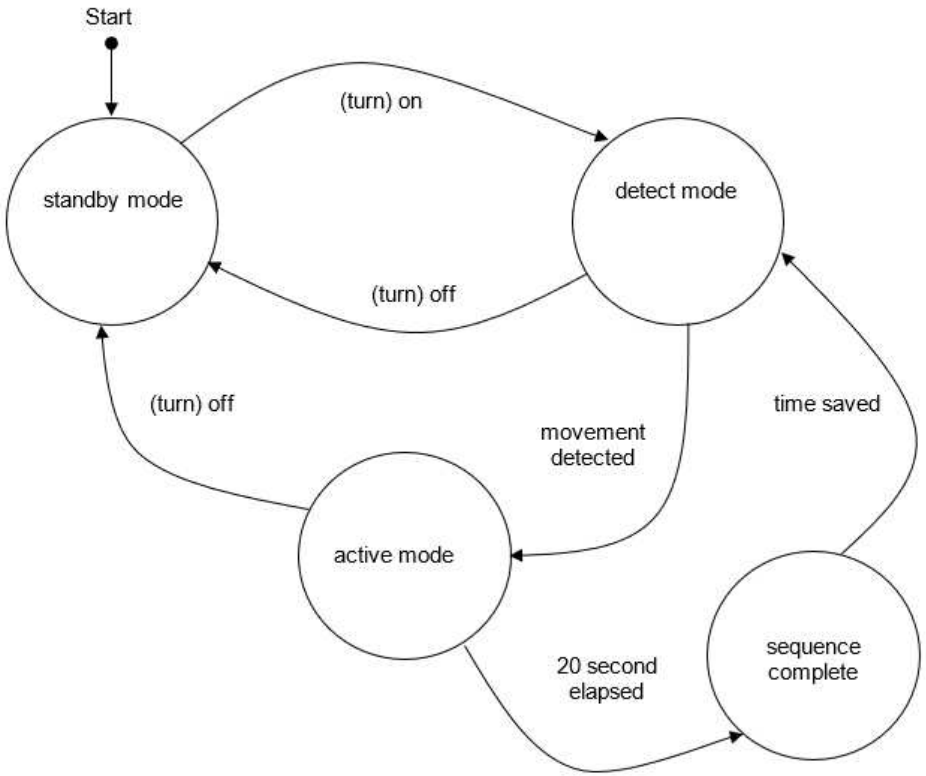
1(d)(ii)	1000	**1**

1(d)(iii)	`DECLARE Product : ARRAY [0:99, 0:9] OF INTEGER` 1 mark for correct upper and lower bound • [0:99, 0:9 1 mark for all other parts of declaration `DECLARE Product : ARRAY [0:99, 0:9] OF INTEGER`	**2**

Examiner report missing
9618_s25_qp_22 Question 1

4	 One mark for each functional group as listed: 1. Initialise Index used for `Number` to either 1 or 0 before first loop 2. Loop 100 times 3. Input value 4. Increment array index **and** store value in `Number` array at element referenced by array `index` **in a loop** 5. Output loop starts at last element in `Number` array 6. Output `Number` array element referenced by `Index` **in a loop** 7. Output all elements of `Number` array **once** in reverse order **Max 6**	**6**

Examiner report missing
9618_s25_qp_22 Question 4

5(a)	 Mark as follows: 1 Mark: The three new states drawn and labelled. 1 Mark: Two of the events drawn with labels connecting correct states and correct line direction 1 Mark: Four of the events drawn with labels connecting correct states and correct line direction. 1 Mark: All and only six events drawn with labels connecting correct states and correct line direction.	**4**

5(b)	Conditional Solution
	
	Example Solution
	
	`DECLARE Count : INTEGER`
	`DECLARE Line : STRING`
	`DECLARE NextHour, Hour : STRING`
	
	`OPENFILE "TimeTaken.txt" FOR READ`
	`Count ← 1`
	`READFILE "TimeTaken.txt", Line`
	`Hour ← LEFT(Line, 2)`
	
	`WHILE NOT EOF("TimeTaken.txt")`
	`  READFILE "TimeTaken.txt", Line`
	`  NextHour ← LEFT(Line, 2)`
	`  IF NextHour = Hour THEN`
	`  Count ← Count + 1`
	`  ELSE`
	`  OUTPUT "Hour : ", Hour, " Total : ", Count`
	`  Hour ← NextHour`
	`  Count ← 1`
	`  ENDIF`
	`ENDWHILE`
	
	`OUTPUT "Hour : ", Hour, " Total : ", Count`
	
	`CLOSEFILE "TimeTaken.txt"`
	
	Mark as follows:
	
	1. Any Initialisation of count for number of pictures taken each hour
	2. Open `"TimeTaken.txt"` for read and subsequently close
	3. Conditional loop until EOF
	4. Read a line from `"TimeTaken.txt"` **in a loop**
	5. Extract Hour from line read **in a loop**
	6. A mechanism to compare current hour extracted from file with last one read from file **in a loop**
	7. If hours same increment count **in a loop**
	8. If **not** same output count (with a suitable message) **and** update `Hour ← NextHour` **and** set Count to 1 **in a loop**
	9. Output final `Count` **and** `Hour` (with a suitable message) **once only**
	
	Note: max 8
		8

Examiner report missing
9618_s25_qp_22 Question 5

6(a)(i)		**4**

Value	Start	Unused	New	Last	Current	
1043	2	8	8	−1	2	Region 1
				2	3	Region 2
				3	1	Region 3
				1	7	
				7	4	Region 4

Award 1 mark per region

6(a)(ii)		**3**

	Data
1	1018
2	1007
3	1010
4	1056
5	1092
6	1062
7	1034
8	**1043**
9	0
10	0

	Pointer
1	**7**
2	**3**
3	**1**
4	**6**
5	**–1**
6	**5**
7	**8**
8	**4**
9	**10**
10	**–1**

Mark as follows:

1 mark for global array `Data` row 8 containing the value 1043

1 mark for global array `Pointer` row 7 containing the value 8 and row 8 containing the value 4

1 mark for all other rows in both arrays

6(b)	Example answer: The ADT is a linked list and the procedure `Place()` inserts / add a new node / value into it / the linked list Mark as follows: 1. 1 mark for identifying the ADT as a linked list 2. 1 mark for identifying operation as inserting / adding a value / node **into a** linked list	**2**

Examiner report missing
9618_s25_qp_22 Question 6

7(a)	Example solution Conditional Loop
	
	`FUNCTION FindCustomer(CustomerID : INTEGER) RETURNS INTEGER`
	
	`DECLARE Index : INTEGER`
	`DECLARE Found : BOOLEAN`
	`CONSTANT Unused = 99999`
	`CONSTANT Upper = 1000`
	`Found ← FALSE`
	
	`Index ← 1`
	
	`IF CustomerID < 10001 OR CustomerID > 11000 THEN`
	`RETURN -1 // Out of range value for customer ID`
	`ENDIF`
	
	`WHILE Found = FALSE AND Loyalty[Index, 1] <> 99999`
	`IF Loyalty[Index,1] = CustomerID THEN`
	`Found ← TRUE`
	`ELSE`
	`Index ← Index + 1`
	`ENDIF`
	`IF Index > Upper THEN`
	`RETURN -1`
	`ENDIF`
	`ENDWHILE`
	
	`IF Found THEN`
	`RETURN Loyalty[Index, 2]`
	`ELSE`
	`RETURN -1`
	`ENDIF`
	
	`ENDFUNCTION`
	
	Mark as follows:
	
	1. Create function header and ending with correct parameter and return type
	2. Check `CustomerID` is in range and if not return `-1`
	3. (Conditional) loop iterating through each element in array
	4. Check for current element in array contains required customer ID **in a loop**
	5. … If found set value to terminate loop
	6. Terminating loop when customer ID found
	7. Terminating loop when current customer ID is 99999
	8. Return either loyalty points for the customer found or `-1` if not found
		8

7(b)	Example solution
	
	`PROCEDURE PointsReport()`
	
	`DECLARE Count : INTEGER`
	`DECLARE Index : INTEGER`
	`DECLARE Sum : INTEGER`
	`DECLARE Average : REAL`
	
	`Index ← 1`
	`Count ← 0`
	`Sum ← 0`
	
	`WHILE Loyalty[Index, 1] <> 99999 AND Index <= 1000`
	`IF Loyalty[Index, 2] >= 11 THEN`
	`OUTPUT Loyalty[Index, 1]`
	`ENDIF`
	`Count ← Count + 1`
	`Sum ← Sum + Loyalty[Index, 2]`
	`Index ← Index + 1`
	`ENDWHILE`
	
	`Average ← Sum / Count`
	`OUTPUT "The average points of all the customers is ", Average`
	
	`ENDPROCEDURE`
	
	Mark as follows:
	
	1. Create procedure header and ending
	2. Declare and initialise `Index`, `Sum` and `Count`
	3. Loop through all elements in `Loyalty` array
	4. … or terminates when column1 of `Loyalty` array equals `99999` **in a loop**
	5. Check if loyalty points greater than or equal to `11` **in a loop**
	6. If true output customer ID
	7 Sum points **and** Increment `Count` **in a loop**
	8 Calculate **and** output average with an appropriate message
	
	Max 7
		7

7(c)(i)	An array can only store data of **the same type** // An array cannot store data **of different types**	**1**

7(c)(ii)	1 mark for each point 1. a new (composite) data type / record is defined (that consists of both `INTEGER` and `STRING`) 2. an array based on this new type is **declared** Alternative 1 mark for each point 1. Converting loyalty points to string and concatenating / joining / append with Customer ID 2. Storing **concatenated** string in an **array of string**	**2**

Examiner report missing
9618_s25_qp_22 Question 7

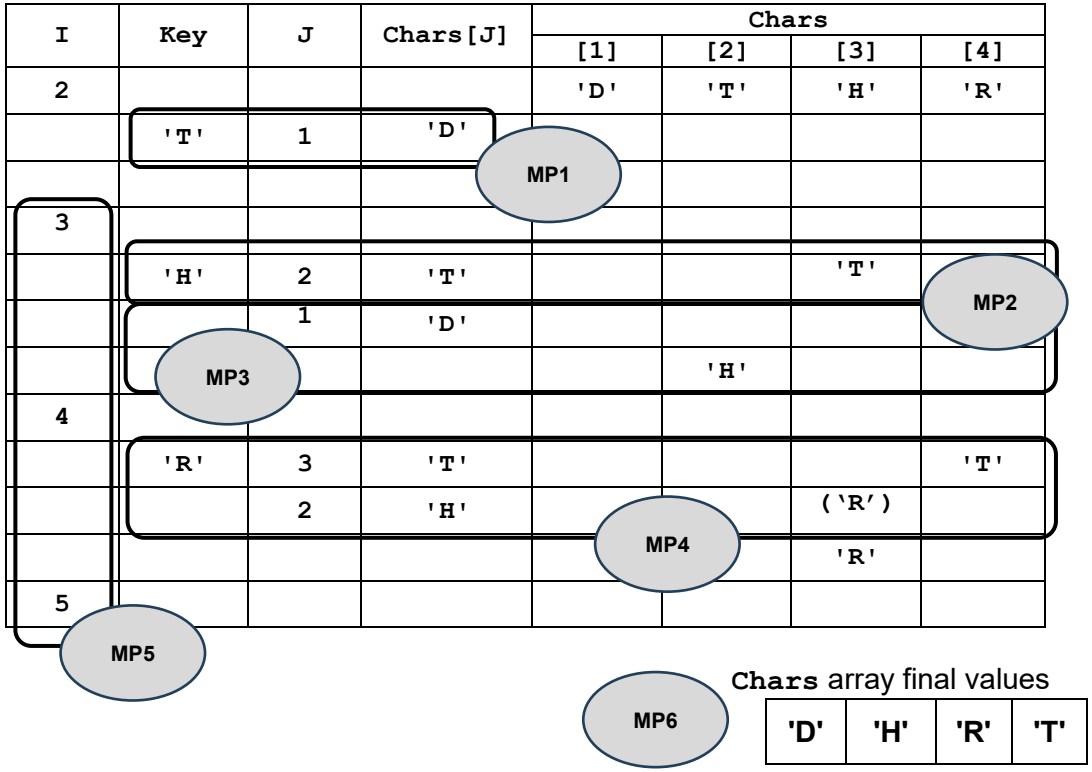
1(a)		**4**

Example value	Explanation	Variable name	Data type
`"Fruit"`	a category of stock that is sold in the shop	**`Category`**	**`STRING`**
`20/02/2025`	when an item was sold	**`DateSold`**	**`DATE`**
`12.67`	the cost of an item	**`(Item)Cost/Price`**	**`REAL`**
`TRUE`	to indicate if an item is in stock	**`(Item)InStock`**	**`BOOLEAN`**

Mark as follows:
1 mark per row for each variable name **and** data type

Examiner report missing
9618_s25_qp_21 Question 1

4(a)	**MP1** Count-controlled **MP2** The number of iterations required is known	**2**

4(b)	 Mark as follows: **MP1, MP2, MP3, MP4, MP5** 1 mark per enclosure	**6**

Examiner report missing
9618_s25_qp_21 Question 4

5	**MP1** **MP2** **MP3** **MP4** **MP5** **MP6** **MP7** **MP8** **MP9** **MP10** **Max 7**	Pop an item off the stack Update Stack pointer Add it to the queue Update Rear pointer Repeat until the **stack is empty** Remove an item from the queue Update Front pointer Push it onto the stack Update the Stack pointer Repeat until the **queue is empty**	**7**

Examiner report missing
9618_s25_qp_21 Question 5

6(a)		**4**

Type of test data	Test data value	Expected outcome
normal	36	data item is accepted
boundary/ extreme/ normal	0 / 1	data item is accepted
boundary/ extreme/ normal	59 / 60	data item is accepted
abnormal	>= 61	data item is rejected
normal	15	data item is accepted

Mark as follows:
1 mark for each row with (Type **and** Value **and** Outcome)

6(b)	Example solution:
	
	`PROCEDURE Sort()`
	`  DECLARE Temp, J, Boundary : INTEGER`
	`  DECLARE NoSwaps : BOOLEAN`
	
	`  Boundary ← 1999`
	
	`  REPEAT`
	`  NoSwaps ← TRUE`
	`  FOR J ← 1 TO Boundary`
	`  IF Reading[J, 1] > Reading[J+1, 1] THEN`
	`  //first swap sensor value`
	`  Temp ← Reading[J, 1]`
	`  Reading[J, 1] ← Reading[J+1, 1]`
	`  Reading[J+1, 1] ← Temp`
	`  //now swap corresponding ID`
	`  Temp ← Reading[J, 2]`
	`  Reading[J, 2] ← Reading[J+1, 2]`
	`  Reading[J+1, 2] ← Temp`
	`  NoSwaps ← FALSE`
	`  ENDIF`
	`  NEXT J`
	`  Boundary ← Boundary - 1`
	`  UNTIL NoSwaps = TRUE`
	
	`ENDPROCEDURE`
	
	MP1 Procedure heading **and** ending
	MP2 Conditional loop correctly formed including Boolean flag declared and initialised
	
	MP3 An inner loop
	MP4 Correct range 1 to 1999 for inner loop
	
	MP5 Comparison of element J with J+1 in a loop
	MP6 Declare `Temp` variable **and** swap elements in a loop
	MP7 **Both** SensorID **and** Speed values were swapped in a loop
	
	MP8 'No-Swap' mechanism:
	• Conditional **outer** loop including flag reset
	• Flag set in **inner** loop to indicate swap
	
	MP9 Reducing `Boundary` in the <u>outer</u> loop
		8

Examiner report missing
9618_s25_qp_21 Question 6

7(a)	Example solution:
	
	`FUNCTION CustomerOrder(Number, Points : INTEGER) RETURNS INTEGER`
	
	`   DECLARE NewPoints, NumberFree: INTEGER`
	
	`   NewPoints ← Points + Number`
	`   NumberFree ← 0`
	`   WHILE NewPoints >= 11 AND Number > 0`
	`      NumberFree ← NumberFree + 1`
	`      NewPoints ← NewPoints -11`
	`       Number ← Number - 1`
	`   END WHILE`
	
	`   OUTPUT "Number of free coffees is: ", NumberFree`
	`   RETURN NewPoints`
	`ENDFUNCTION`
	
	MP1 Function header **and** ending **and** parameters **and** return type
	MP2 Number of coffees ordered added to points
	
	MP3 Correct calculation of one free coffee
	
	MP4 Attempted calculation of multiple free coffees **and** reduced `NewPoints`
	
	MP5 Output number of free drinks with suitable message
	
	MP6 Return of `NewPoints`
		6

7(b)(i)	Example solution:
	
	`PROCEDURE AddNewCustomers(NumToAdd : INTEGER)`
	
	`   DECLARE CustomerID : INTEGER`
	`   DECLARE Count : INTEGER`
	`   DECLARE Line : STRING`
	`   DECLARE NewLine : STRING`
	`   OPENFILE "Loyalty.txt" FOR READ`
	
	`   WHILE NOT EOF("Loyalty.txt")`
	`      READFILE "Loyalty.txt", Line`
	`   ENDWHILE`
	`   CLOSEFILE "Loyalty.txt"`
	
	`   OPENFILE "Loyalty.txt" FOR APPEND`
	
	`   CustomerID ← STR_TO_NUM((LEFT(Line, 6))`
	
	`   FOR Count ← 1 TO NumToAdd`
	`      CustomerID ← CustomerID + 1`
	`      OUTPUT CustomerID`
	`      NewLine ← NUM_TO_STR(CustomerID) & ",0"`
	`      WRITEFILE "Loyalty.txt", NewLine`
	`   NEXT Count`
	
	`   CLOSEFILE "Loyalty.txt"`
	`ENDPROCEDURE`
	
	Mark as follows:
	MP1 All variables used are declared using the correct type including a string **and** an integer
	MP2 Open file in read mode **and** close (before opening in append mode)
	
	MP3 Conditional loop with `EOF("Loyalty.txt")`
	MP4 Read `Line` from file // Count number of records in the file
	
	MP5 Open the file in append mode **and** subsequently close
	MP6 Extract `CustomerID` from `Line` **and** convert to integer // use count from MP4 to generate last `CustomerID` stored
	MP7 A (count controlled) loop for the number of customers to add
	MP8 Increment `CustomerID` **and** output the new `CustomerID` in loop
	
	MP9 Create string for new `CustomerID` **and** write to file in loop
	
	Max 8
		8

7(b)(ii)	**MP1** **MP2** **MP3** **MP4** **MP5** **Max 4**	Check if the text file `loyalty.txt` exists / is empty If file does not exist it must be created (using write mode) If the file has to be created / is empty then set the first `CustomerID` to `100001` Write `"100001,0"` to `loyalty.txt` (using write mode) For all but the first customer (to be added to the empty file) **use the existing code / module**	**4**

Examiner report missing
9618_s25_qp_21 Question 7

1(b)		**3**

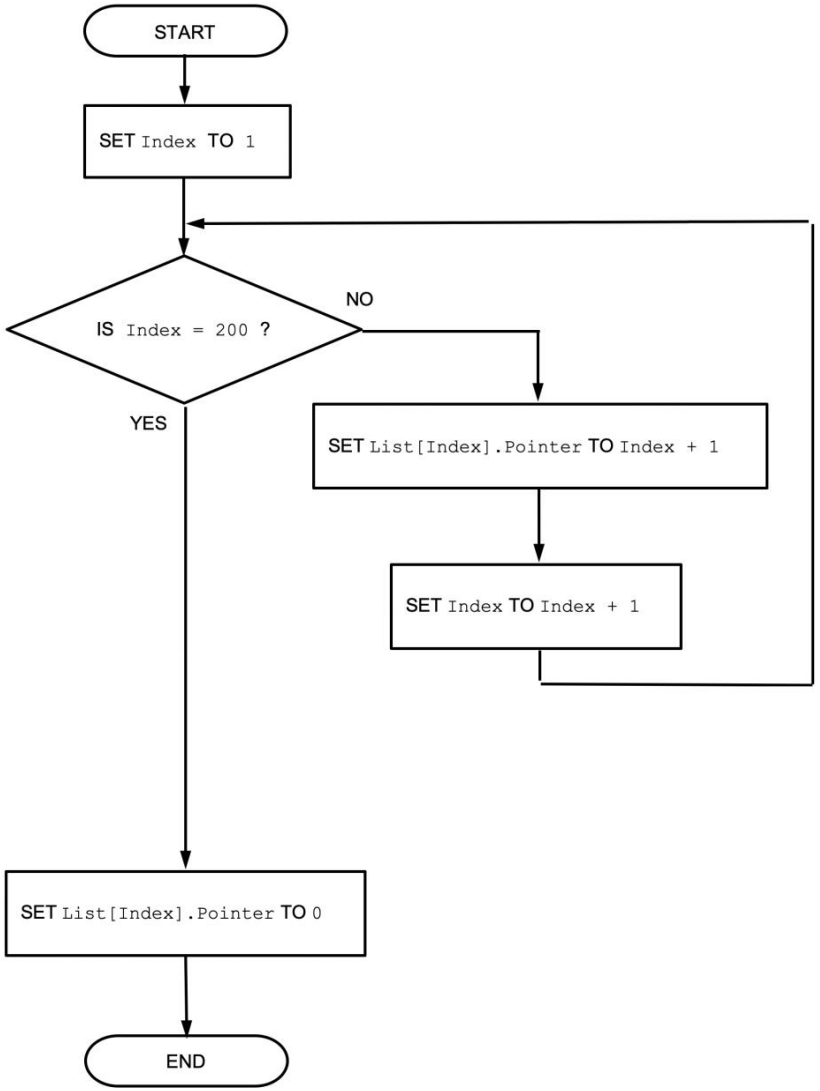
Variable	Example data value	Data type
`Available`	`TRUE`	BOOLEAN
`Received`	`"18/04/2021"`	STRING
`Index`	`100`	INTEGER

One mark per row

Examiner report missing
9618_w24_qp_23 Question 1

3(a)(i)	0 is not a valid array / `List` index value // No 0th element in the array	**1**

3(a)(ii)	`0` to `200`	**1**

3(b)	 Mark as follows: **MP1** Use of variable as array index **and** initialisation to 1 / to first element of array **MP2** Loop for 199 / 200 iterations **MP3** Assign Index+1 to each pointer field in a loop **MP4** Assign 0 to 200th pointer field	**4**

3(c)	One mark per step: **MP1** Assign `HeadPointer` to `ThisPointer` / Current Pointer // Identify first node using headpointer **MP2** Loop the following until `ThisPointer` value is 0 (zero) / null pointer reached **MP3** … Output data (field) from array element at index `ThisPointer` // Output data (field) of the current node / array element **MP4** … Assign pointer field from current array element / index / to `ThisPointer` / Current Pointer // Assign pointer field from current node to `ThisPointer` / Current Pointer	**4**

Examiner report missing
9618_w24_qp_23 Question 3

4(a)	One mark per highlighted part:
	
	`FOR Index ← 1 TO 5`
	`  Lower ← GB[Index] - 2`
	`  Upper ← GB[Index] + 2  // Lower + 4`
	`  IF Mark >= Lower AND Mark <= Upper THEN`
	`//IF Mark <= Upper AND Mark >= Lower THEN`
	`  OUTPUT "Check this paper"`
	`  ENDIF`
	`NEXT Index`
		4

4(b)(i)	**MP1** Use `Mark` as the index to the `Check` array // to specify an array element **MP2** If value of indexed element is `TRUE`, then the paper will need to be checked	**2**

4(b)(ii)	Example:solution
	
	`PROCEDURE GBInitialise()`
	`  DECLARE GBIndex, GBMark, ThisIndex : INTEGER`
	
	`  FOR ThisIndex ← 0 TO 75`
	`  Check[ThisIndex] ← FALSE // initialise all values`
	`  NEXT ThisIndex`
	
	`  FOR GBIndex ← 1 TO 5`
	`  GBMark ← GB[GBIndex]`
	`  FOR ThisIndex ← GBMark - 2 TO GBMark + 2`
	`  Check[ThisIndex] ← TRUE //Set GBMark ± 2 to TRUE`
	`  NEXT ThisIndex`
	`  NEXT GBIndex`
	`ENDPROCEDURE`
	
	Mark as follows:
	MP1 Procedure heading and ending
	MP2 Initialise `Check` array to `FALSE`
	MP3 Loop through `GB` array
	MP4 Extract the `GB` mark
	MP5 Loop for 5 elements across `GBMark` ± 2 // Check if within `GBMark` ± 2
	MP6 Assign each element of `Check` array to `TRUE`
	
	ALTERNATIVE – Example using selection Version 1
	
	`FOR ThisIndex ← 0 TO 75`
	`  CASE ThisIndex`
	`  GB[1] - 2 TO GB[1] + 2 : Check[ThisIndex] ← TRUE`
	`  GB[2] - 2 TO GB[2] + 2 : Check[ThisIndex] ← TRUE`
	`  GB[3] - 2 TO GB[3] + 2 : Check[ThisIndex] ← TRUE`
	`  GB[4] - 2 TO GB[4] + 2 : Check[ThisIndex] ← TRUE`
	`  GB[5] - 2 TO GB[5] + 2 : Check[ThisIndex] ← TRUE`
	`  OTHERWISE: Check[ThisIndex] ← FALSE`
	`  ENDCASE`
	`NEXT ThisIndex`
		6

Examiner report missing
9618_w24_qp_23 Question 4

8(a)	Example solution
	
	`PROCEDURE Count(ThisPlayer : INTEGER, ThisRole : STRING)`
	`  DECLARE Index, Num, Total : INTEGER`
	
	`  Num ← 0`
	`  Total ← 0`
	
	`  FOR Index ← 1 TO 45`
	`  IF Character[Index].Player = ThisPlayer AND __`
	`  Character[Index].Role = ThisRole THEN`
	`  Num ← Num + 1`
	`  Total ← Total + Character[Index].SkillLevel`
	`  NEXT Index`
	
	`  IF Num > 0 THEN`
	`  OUTPUT "Player ", ThisPlayer, __`
	`  " has ", Num, " characters with the role of`
	`  ", ThisRole, " and the total skill level is`
	`  ", Total`
	`  ELSE`
	`  OUTPUT "No characters with that role are`
	`  assigned to this player"`
	`  ENDIF`
	
	`ENDPROCEDURE`
	
	MP1 Initialisation of local integers for `Num` and `Total`
	MP2 Loop through 45 elements
	
	MP3 Attempt to check Player and Role fields **in a loop**
	MP4 Correctly compare `Player` field with parameter **in a loop**
	MP5 Correctly compare `Role` field with parameter **in a loop**
	
	MP6 ... if player and role found, increment `Num` **and** sum Skill Total **in a loop**
	MP7 Test for any matches **after the loop**
	MP8 **Both** possible OUTPUT statements correctly formed following an attempt at MP6 but outputting one only
	
	Max 7 marks
		7

8(b)	Example solution:
	
	`PROCEDURE Restore()`
	`  DECLARE Index : INTEGER`
	`  DECLARE Line  : STRING`
	
	`  OPENFILE "SaveFile.txt" FOR READ`
	`  FOR Index ← 1 TO 45`
	`  READFILE "SaveFile.txt", Line`
	`  Character[Index].Player ← STR_TO_NUM(Extract(Line, 1))`
	`  Character[Index].Role ← Extract(Line, 2)`
	`  Character[Index].Name ← Extract(Line, 3)`
	`  Character[Index].Skill ← STR_TO_NUM(Extract(Line, 4))`
	`  NEXT Index`
	
	`  CLOSEFILE "SaveFile.txt"`
	
	`ENDPROCEDURE`
	
	Mark as follows:
	MP1 Open the file in read mode and subsequently close
	MP2 Loop through 45 elements
	MP3 Read a line from the file **in a loop**
	MP4 Attempt to use `Extract()` **in a loop**
	MP5 Correct use of `Extract()` for all fields **in a loop**
	MP6 Use of `STR_TO_NUM()` on Player and Skill **in a loop**
	MP7 Completely correct extraction and assignment of all fields **in a loop**
		7

8(c)	**MP1** The `Character` array / character data can be saved before the program is closed **MP2** Allowing the **game to continue** using the same data / from the point it was saved	**2**

Examiner report missing
9618_w24_qp_23 Question 8

1(a)	(Corrective) Maintenance	**1**

1(c)	Features include: **MP1** Editor **MP2** Auto- (syntax) complete / auto correction // identify undeclared variables(s) **MP3** Prettyprint / auto-indentation / auto (structure) highlighter **MP4** Dynamic syntax checking **MP5** Expand / collapse code blocks **MP6** Context sensitive prompts **Max 2 marks**	**2**

1(d) One mark per row: **3**

Variable name	Used to store	Data type
`Name`	a customer name	**STRING**
`Index`	an array index	**INTEGER**
`Result`	the result of the division of any two non-zero numbers	**REAL**

Examiner report missing
9618_w24_qp_22 Question 1

2(a)	Example 'loop solution':
	
	`FUNCTION Conceal(CardNumber : STRING) RETURNS STRING`
	`  DECLARE MaskedString : STRING`
	`  DECLARE Count : INTEGER`
	`  CONSTANT Asterisk = '*'`
	
	`  MaskedString ← RIGHT(CardNumber, 4)`
	`  FOR Count ← 1 TO LENGTH(CardNumber) - 4`
	`  MaskedString ← Asterisk & MaskedString`
	`  NEXT Count`
	
	`  RETURN MaskedString`
	
	`ENDFUNCTION`
	
	Mark as follows:
	MP1 Function heading **and** parameter **and** ending **and** return type
	MP2 Declaration of all local variables used - including the loop counter
	MP3 Calculate number of digits to mask / number of asterisks required
	
	MP4 use of a 'relevant' Loop
	MP5 Correct number of iterations
	MP6 Concatenate asterisk to start/end of `MaskedString` **in a loop**
	
	MP7 Assign last four characters to `MaskedString //` Concatenate the retained original last four digits
	MP8 Return masked string
	
	Max 6 marks
	
	ALTERNATIVE 'non loop' solution:
	
	`FUNCTION Conceal(CardNumber : STRING) RETURNS STRING`
	`  DECLARE MaskedString : STRING`
	`  DECLARE Count : INTEGER`
	`  CONSTANT Asterisks = "********************" //20 asterisks`
	
	`  Count ← LENGTH(CardNumber) - 4`
	`  MaskedString ← LEFT(Asterisks, Count) & RIGHT(CardNumber, 4)`
	
	`  RETURN MaskedString`
	
	`ENDFUNCTION`
	
	Mark as follows:
	MP1 Function heading **and** parameter **and** ending **and** return type
	MP2 Declaration of all local variables used
	MP3 Calculate number of digits to mask / number of asterisks required
		6

2(b)(i)	`DECLARE CardNumber : ARRAY [1:100, 1:2] OF STRING` **MP1** Correct dimensions **MP2** All other parts of the statement correct	**2**

Examiner report missing
9618_w24_qp_22 Question 2

3(a)(i)	`SP`: 1 `OnStack`: 0 One mark for both correct values	**1**

3(a)(ii)	**MP1** Unused values cannot be popped / taken off the stack // initialised values would never be used / unused elements cannot be accessed **MP2** ... until a value has first been pushed / written // overwrites previous value	**2**

3(b)	Example solution:
```
FUNCTION Push(ThisValue : REAL) RETURNS BOOLEAN
   DECLARE ReturnValue : BOOLEAN
   IF OnStack = 60 / >59 // SP = 61 / SP > 60 //
                                    SP outside the range 1 to 60 THEN
      RETURN FALSE // Stack is already full
   ENDIF
   ThisStack[SP] ← ThisValue
   SP ← SP + 1
   OnStack ← OnStack + 1
   RETURN TRUE
ENDFUNCTION1
```

Mark as follows:
One mark per gap | **4** |

Examiner report missing
9618_w24_qp_22 Question 3

6(a)	Example solution:
	
	`PROCEDURE Special()`
	`  DECLARE Index : INTEGER`
	`  DECLARE Filling1, Filling2 : STRING`
	
	`  REPEAT`
	`  Index ← INT(RAND(35)) + 1`
	`  UNTIL Filling[Index] <> ""`
	
	`  Filling1 ← Filling[Index]`
	
	`  REPEAT`
	`  Index ← INT(RAND(35)) + 1`
	`  UNTIL Filling[Index] <> "" AND Filling1 <> Filling[Index]`
	
	`  Filling2 ← Filling[Index]`
	
	`  REPEAT`
	`  Index ← INT(RAND(10) + 1)`
	`  UNTIL Bread[Index] <> ""`
	
	`  OUTPUT "The daily special is ", Filling1, " and ", __ Filling2, " on ", Bread[Index], " bread."`
	
	`ENDPROCEDURE`
	
	Mark as follows:
	MP1 Loop for Filling 1, avoiding unused elements
	MP2 Loop for Filling 2 avoiding unused elements
	MP3 Check Filling 2 is different from Filling 1 – could correctly compare either the indices or the array contents
	
	MP4 Loop for Bread, avoiding unused elements
	
	MP5 Using `RAND(10) / RAND(35)`
	MP6 Completely correct use of `RAND()` – including `INT()` and +1 in all cases
	
	MP7 Correct output - **once only** – following a reasonable attempt at selection of filings and bread
		7

Examiner report missing
9618_w24_qp_22 Question 6

8(a)	Example solution:
	
	`PROCEDURE Assign(ThisRole : STRING, ThisPlayer : INTEGER)`
	`  DECLARE Index : INTEGER`
	`  DECLARE Done : BOOLEAN`
	
	`  Done ← FALSE`
	`  Index ← 1`
	
	`  WHILE Index < 46 AND Done = FALSE`
	`  IF Character[Index].Player = 0 AND __`
	`  Character[Index].Role = ThisRole THEN`
	`  Character[Index].Player ← ThisPlayer`
	`  Done ← TRUE`
	`  ELSE`
	`  Index ← Index + 1`
	`  ENDIF`
	`  ENDWHILE`
	
	`  IF Done = TRUE THEN`
	`  OUTPUT Character[Index].Name, " the ",__`
	`  Character[Index].Role, __`
	`  " has been assigned to player ", ThisPlayer`
	`  ELSE`
	`  OUTPUT "No characters with this role are available"`
	`  ENDIF`
	
	`ENDPROCEDURE`
	
	Mark as follows:
	MP1 Loop until 'found' or all 45 elements considered
	MP2 Test of `Player` field – i.e. not value **in a loop**
	
	MP3 ... `AND Role` **–** i.e. match for **`ThisRole`** parameter **in a loop**
	
	MP4 If available character found, assign `ThisPlayer` to the character **in a loop**
	MP5 When character found set termination condition/flag
	MP6 **Both** OUTPUT messages logically correctly placed
	
	MP7 **Both** OUTPUT statements correctly formed
		7

8(b)	Example solution:
	
	`PROCEDURE Save()`
	`  DECLARE Index  : INTEGER`
	`  DECLARE Line  : STRING`
	`  CONSTANT SEP = '^'`
	
	`  OPENFILE "SaveFile.txt" FOR WRITE`
	`  FOR Index ← 1 TO 45`
	`  Line ← NUM_TO_STR(Character[Index].Player) & SEP`
	`  Line ← Line & Character[Index].Role & SEP`
	`  Line ← Line & Character[Index].Name & SEP`
	`  Line ← Line & NUM_TO_STR(Character[Index].Level)`
	`  WRITEFILE "SaveFile.txt", Line`
	`  NEXT Index`
	
	`  CLOSEFILE "SaveFile.txt"`
	`ENDPROCEDURE`
	
	Mark as follows:
	MP1 Declaration of local integer for `Index` (**and** string type for `Line`)
	MP2 Open `"SaveFile.txt"` in write mode **and** subsequently close
	MP3 Loop through 45 elements
	
	MP4 Attempt to form `Line` - four fields **in a loop**
	MP5 Correct use of `NUM_TO_STR()x2 (Player and Level)` **in a loop**
	MP6 Correct use of **three** &<separator>& `strings` **in a loop**
	
	MP7 Line from MP4 written to file **in a loop**
		7

8(c)	**MP1** Encode `Status` as a character / string **MP2** Append the '^' separator and the character/string	**2**

8(d)	**MP1** Method: Create a filename **suffix** which is **incremented** for each file save **MP2** Example: SaveFile01.txt, SaveFile02.txt	**2**

Examiner report missing
9618_w24_qp_22 Question 8

1(a)	One mark per point: Error: The **calculation** of TaxPayable is incorrect Correction: TaxPayable ← (ItemCost * TaxRate) / 100	**2**

1(c)	One mark per row: **Variable name** / **Data type** HighRate / Boolean TaxPayable / Real	**2**

Examiner report missing
9618_w24_qp_21 Question 1

3(a)	One mark per emboldened part: 1 Open the file **(Result.txt) in <u>read</u> mode** 2 Loop until **EOF(Result.txt)** // **EOF** // **end of file** 3 **<u>Read a</u> / <u>the</u> / <u>next line</u> (from the file)** and store in `ThisLine` 4 Assign `LineY` to `LineX` 5 Assign `LineZ` to `LineY` 6 Assign `ThisLine` to `LineZ` 7 After the loop, **close the file (Result.txt)** 8 Output `LineX`, `LineY`, `LineZ`	**4**

3(c)	Two loop solution One mark per point: 1 Loop until given line number (-1) / `<parameter> (-1)` (lines have been read) 2 ... if end of file is reached then return FALSE 3 Loop for three lines // Read three lines // Repeat of 4 and 5 for three lines 4 Read a line and output it 5 ... if end of file is reached then return FALSE 6 After outputting the (required) lines return TRUE 7 Ordering of lines no longer needed **Max 4 marks** Alternative solution **One loop solution mark scheme that reads to given line number + 2** One mark per point 1 Loop until given line number + 2 / `<parameter> + 2` (lines have been read) 2 **OR** end of file is reached 3 Return FALSE if EOF reached 4 Read a line from the file **in the loop** 5 Continue to order the last three lines read i**n the loop** // Steps 4 to 6 stay the same 6 (If FALSE has **not** been returned) output the required three lines (in the correct order) 7 … **and** Return TRUE **Max 4 marks**	**4**

Examiner report missing
9618_w24_qp_21 Question 3

4(a)	One mark per point: **Type:** Run-time error **Cause:** A divide by zero operation is attempted // Attempt to convert a non-numeric string to a number	**2**

4(c)	Example solution:
	
	`FUNCTION Evaluate(NumStr1, NumStr2 : STRING) RETURNS Result`
	`  DECLARE RetVal : Result`
	`  DECLARE Num1, Num2 : REAL`
	
	`  RetVal.Done ← FALSE`
	
	`  IF IS_NUM(NumStr1) = TRUE AND IS_NUM(NumStr2) = TRUE THEN`
	`  Num1 ← STR_TO_NUM(NumStr1)`
	`  Num2 ← STR_TO_NUM(NumStr2)`
	`  IF Num2 <> 0 THEN`
	`  RetVal.Value ← (Num1 / Num2)`
	`  RetVal.Done ← TRUE`
	`  ENDIF`
	`  ENDIF`
	
	`RETURN RetVal`
	
	`ENDFUNCTION`
	
	Note that order of parameters is not specified so divisor could be either parameter.
	
	Mark as follows:
	
	1 Function heading, parameters, ending
	2 ... Correct return type of `Result`
	3 Local declaration of type `Result`
	4 Attempt at using `IS_NUM()` to check **both** parameters/strings
	5 Attempt at using of `STR_TO_NUM()` to convert **both** parameters/strings
	6 Check if the divisor, is non-zero ...
	7 … **and** if so correct calculation **and** assignment to `Value` field
	8 Return variable of type `Result` having assigned both fields
	
	Max 6 marks
		6

Examiner report missing
9618_w24_qp_21 Question 4

6(a)	One mark per point: 1 Fewer lines of code are needed 2 The program is simpler/ less complex // Program is easier to design / code / maintain / modify / test / debug 3 Direct access to days in a month / data (using month number as index) // Can use index / month to (directly) access days in month / data **Max 2 marks**	**2**

6(b)	Example Solution
	
	`FUNCTION GetDate(ProductionDate : DATE, ShelfLife : INTEGER) RETURNS DATE`
	`  DECLARE NewDate : DATE`
	`  DECLARE DD, MM, YY, LastDay : INTEGER`
	
	`  DD ← DAY(ProductionDate)`
	`  MM ← MONTH(ProductionDate)`
	`  YY ← YEAR(ProductionDate)`
	
	`  DD ← DD + ShelfLife`
	`  LastDay ← DaysInMonth[MM]`
	
	`  IF DD > LastDay THEN`
	`  MM ← MM + 1`
	`  DD ← DD – LastDay`
	`  IF MM = 13 THEN`
	`  MM ← 1`
	`  YY ← YY + 1`
	`  ENDIF`
	`  ENDIF`
	`  NewDate ← SETDATE(DD, MM, YY)`
	`  RETURN NewDate`
	
	`ENDFUNCTION`
	
	Mark as follows:
	
	1 Declare three variables of `TYPE INTEGER` to store `DD`, `MM` and `YY`
	2 Correct use of date functions from Insert to extract three integer values representing DD, MM and YY and use/assign to a variable
	3 Add parameter `ShelfLife` to `DD` and use / assign result to a variable
	4 Extract `LastDay` from `DaysInMonth` using `MM` as `Index`
	5 Test for new `DD` after end of month / `DaysInMonth[MM]`
	… and if `DD` > `DaysInMonth[MM]`:
	6 `MM` is incremented by 1 and `DD` is set to `DD – DaysInMonth[MM]`
	7 If `MM = 13 / MM > 12` then increment `YY` **and** set `MM` to 1
	8 Use of `SETDATE` to assign value to `NewDate`, `NewDate` must have been delared as a type `DATE`
	9 Return date
	
	Max 7 marks
		7

Examiner report missing
9618_w24_qp_21 Question 6

8(a)	Loop example solution
	
	`FUNCTION CheckMark(Mark : INTEGER) RETURNS BOOLEAN`
	`  DECLARE Index, Lower, Upper : INTEGER`
	
	`  FOR Index ← 1 TO 5`
	`  Lower ← GradeBoundary[Index] - 2`
	`  Upper ← GradeBoundary[Index] + 2`
	`  IF Mark >= Lower AND Mark <= Upper THEN`
	`  RETURN TRUE`
	`  ENDIF`
	`  NEXT Index`
	
	`  RETURN FALSE`
	`ENDFUNCTION`
	
	Mark as follows:
	1 Loop through all elements in `GradeBoundary` array
	2 Attempt to calculate range, both `Lower` and `Upper`, **in a loop**
	3 Completely correct range calculation **in a loop**
	4 Test if given mark / parameter, is within range **in a loop**
	5 Set variable / immediate RETURN if recheck required **in a loop**
	6 Return Boolean in both cases following a reasonable attempt
	
	Selection example solution
	
	`FUNCTION CheckMark(Mark : INTEGER) RETURNS BOOLEAN`
	
	`  CASE OF Mark`
	`  GradeBoundary[1] - 2 TO GradeBoundary[1] + 2 :`
	`  RETURN TRUE`
	`  GradeBoundary[2] - 2 TO GradeBoundary[2] + 2 :`
	`  RETURN TRUE`
	`  GradeBoundary[3] - 2 TO GradeBoundary[3] + 2 :`
	`  RETURN TRUE`
	`  GradeBoundary[4] - 2 TO GradeBoundary[4] + 2 :`
	`  RETURN TRUE`
	`  GradeBoundary[5] - 2 TO GradeBoundary[5] + 2 :`
	`  RETURN TRUE`
	`  OTHERWISE : RETURN FALSE`
	`  ENDCASE`
	
	`ENDFUNCTION`
	
	Mark as follows:
	1 Correct syntax used for selection structure(s)
	2 Correct checks for two ranges
	3 Correct checks for three ranges
	4 Correct checks for four ranges
	5 Correct checks for all five ranges
	6 Return Boolean in both cases following a reasonable attempt
		6

8(b)	Example Solution:
	
	`PROCEDURE CheckAll(CNum : INTEGER)`
	`  DECLARE Index, Count, ThisMark: INTEGER`
	
	`  Count ← 0`
	`  OPENFILE "GRList.txt" FOR WRITE`
	`  FOR Index ← 1 to CNum`
	`  ThisMark ← Result[Index, 1] // 2D array: mark + ID`
	`  IF CheckMark(ThisMark) = TRUE THEN`
	`  WRITE "GRList.txt", NUM_TO_STR(Result[Index, 2])`
	`  Count ← Count + 1`
	`  ENDIF`
	`  NEXT Index`
	`  CLOSEFILE "GRList.txt"`
	`  OUTPUT "There are ", Count, " papers to check"`
	
	`ENDPROCEDURE`
	
	Mark as follows:
	1 Procedure heading, parameter and ending.
	2 Open file in write mode and subsequently close
	3 Loop through all `CNum` candidates
	4 Extract candidate mark from `Result` array **in a loop**
	5 Use of `CheckMark()` with candidate mark as parameter **in a loop**
	6 Test return value and if `TRUE` then increment `Count` **in a loop**
	7 ... write ID to file ...
	8 ... after conversion to a string **in a loop**
	9 Final output of message giving number of papers to check **not in a loop**
	
	Max 8 marks
		8

8(c)	The file will need to be opened in <u>APPEND</u> mode	**1**

Examiner report missing
9618_w24_qp_21 Question 8

Examiner report missing
9618_w24_qp_21 Question 82

Examiner report missing
9618_w24_qp_21 Question 83

Examiner report missing
9618_w24_qp_21 Question 84

Examiner report missing
9618_w24_qp_21 Question 85

2(a)	One mark per point: 1. Open the file (in read mode and subsequently close) 2. Initialise an index variable (to 1 // 0) 3. Repeat (the next three steps) until the end of file is reached 4. Read a line from the file (into a string variable) 5. Store the line / variable in the array at the index 6. Increment the index **in a loop** **Note: max 5 marks**	**5**

Examiner report missing
9618_s24_qp_23 Question 2

3(a)(i)	Example solution using AND
	
	`IF Batch[ThisIndex].Weight >= Min AND`
	`Batch[ThisIndex].Weight <= Max THEN`
	
	Alternative solution using OR
	
	`IF Batch[ThisIndex].Weight < Min OR`
	`Batch[ThisIndex].Weight > Max THEN`
	
	Mark as follows:
	1. Reference to `Batch[ThisIndex].Weight`
	2. A valid check for one boundary
	3. A valid check for other boundary with correct logic operator
		3

3(a)(ii)	One mark for either: • Set the `Item_ID` field to an empty string / NULL / invalid value • Set `Weight` to <= 0 / zero	**1**

Examiner report missing
9618_s24_qp_23 Question 3

4(a)	Example solution:
	
	`PROCEDURE TwoParts()`
	`  DECLARE NextNum, Average : REAL`
	
	`  TotalA ← 0.0 // 0`
	`  TotalB ← 0.0 // 0`
	
	`  REPEAT`
	`  INPUT NextNum`
	`  TotalA ← TotalA + NextNum`
	`  UNTIL NextNum = 0`
	
	`  REPEAT`
	`  INPUT NextNum`
	`  TotalB ← TotalB + NextNum`
	`  UNTIL NextNum = 0`
	
	`  Average ← (TotalA + TotalB) / 2`
	`  OUTPUT "The average is ", Average`
	`ENDPROCEDURE`
	
	Mark as follows:
	1. Procedure heading and ending
	2. Declare all local variables
	3. Initialise `TotalA` and `TotalB`
	4. First conditional loop until zero entered, summing `TotalA` // Loop until both parts (sequences) have been entered
	5. Second conditional loop until zero entered, summing `TotalB` // Loop summing appropriate Totals
	6. Calculation of average and output with a message // Calculation of the average for the values making up the two totals and both output with a suitable message
		6

4(b)(i)	(1D) array of 20 reals Marks as follows: 1 mark for array 1 mark for 20 reals	**2**

4(b)(ii)	One mark per point: 1 (Multiple instances referenced via a single identifier so) **fewer identifiers needed** 2 Easier to process / search / organise / access the data // Values may be accessed via a loop-controlled variable /an index //An array / data can be iterated through 3 Makes the program/algorithm easier to write / design / understand / maintain / modify // Simplifies the program // Easier to debug / test	**3**

Examiner report missing
9618_s24_qp_23 Question 4

8(a)	Example:
	
	`FUNCTION Header(Line : STRING) RETURNS STRING`
	
	`   IF LENGTH(Line) >= 13 THEN`
	`      IF TO_UPPER(LEFT(Line, 9)) = "FUNCTION " THEN`
	`         RETURN "F" & RIGHT(Line, LENGTH(Line) - 9)`
	`      ENDIF`
	
	`      IF TO_UPPER(LEFT(Line, 10)) = "PROCEDURE " THEN`
	`         RETURN "P" & RIGHT(Line, LENGTH(Line) - 10)`
	`      ENDIF`
	`   ENDIF`
	
	`   RETURN ""`
	
	`ENDFUNCTION`
	
	Mark as follows:
	
	1 Function heading, parameter, return type and ending
	2 Check that the line is at least 13 characters long before attempting to extract and return empty string
	3 Attempt at: Extract characters, 9 or 10 characters, corresponding to keyword plus space and compare with appropriate keyword plus space
	4 Completely correct MP3
	5 Use of type case conversion to allow for 'any case'
	6 Calculation of 'rest of line' and concatenation with 'P or 'F'
	7 Return string
		7

8(b)	Example:
	
	`PROCEDURE FindModules(FileName : STRING)`
	`  DECLARE Line : STRING`
	`  DECLARE Index, LineNum : INTEGER`
	
	`  OPENFILE FileName FOR READ`
	
	`  Index ← 1`
	`  LineNum ← 0`
	
	`  WHILE NOT EOF(FileName)`
	`  READFILE FileName, Line`
	`  LineNum ← LineNum + 1`
	`  Line ← Header(Line)`
	`  IF Line <> "" THEN`
	`  ModInfo[Index, 1] ← NUM_TO_STR(LineNum)`
	`  ModInfo[Index, 2] ← LEFT(Line, 1)`
	`  ModInfo[Index, 3] ← RIGHT(Line, LENGTH(Line) - 1)`
	`  Index ← Index + 1`
	`  ENDIF`
	`  ENDWHILE`
	
	`  CLOSEFILE FileName`
	
	`ENDPROCEDURE`
	
	Mark as follows:
	
	1 Open file in `READ` mode **and** subsequently close
	2 Loop to `EOF(FileName)`
	3 Read a line from the file and maintain `LineNum` in **a loop**
	4 Call `Header()` and use return value **in a loop**
	5 Test return value for "" **in a loop**
	6 Attempt at all three-array assignment for all columns in correct row
	7 Correct values assigned to all columns of array
	8 Maintain correct array row index
		8

Examiner report missing
9618_s24_qp_23 Question 8

1(a)		**4**

Pseudocode example	Selection	Iteration	Input/Output

`FOR Index ← 1 TO 10`
`  Data[Index] ← 0`
`NEXT Index`
		✓	

`WRITEFILE ThisFile, "*****"`			✓
`UNTIL Level > 25`		✓	

`IF Mark > 74 THEN`
`  READFILE OldFile, Data`
`ENDIF`
	✓		✓

One mark per row.

1(b)	**Expression**
	`MyInt ← INT (3.1415926)`
	`MyChar ← MID ("Elwood", 3, 1)`
	Any of:
	• `MyString ← NUM_TO_STR (INT (27.509))`
	• `MyString ← CHR (INT (27.509))`
	• `MyString ← TO_UPPER( NUM_TO_STR( 27.509))`
	• `MyString ← TO_LOWER( NUM_TO_STR( 27.509))`
	Any of:
	• `MyInt ← STR_TO_NUM ( RIGHT ("ABC123", 3))`
	• `MyInt ← LENGTH ( RIGHT ("ABC123", 3))`
	• `MyInt ← LENGTH ( LEFT ("ABC123", 3))`
	One mark per row
		4

Examiner report missing
9618_s24_qp_22 Question 1

3(a)(i)	Pseudocode:
	
	`TYPE Component`
	`  DECLARE Item_Num : INTEGER`
	`  DECLARE Reject  : BOOLEAN`
	`  DECLARE Stage  : CHAR`
	`  DECLARE Limit_1  : REAL`
	`  DECLARE Limit_2  : REAL`
	`ENDTYPE`
	
	Mark as follows:
	
	1 One mark for `TYPE` and `ENDTYPE` statements
	2 One mark for `Item_Num` **and** `Reject` fields
	3 One mark for `Stage` field
	4 One mark for Limit fields as `REAL`
		4

3(a)(ii)	`DECLARE Item : ARRAY [1:2000] OF Component//`
	`DECLARE Item : ARRAY [2000] OF Component//`
	`DECLARE Item : ARRAY [0:1999] OF Component`
	
	One mark per underlined phrase
		2

3(b)	One mark per point: 1 Allows for iteration / can use a loop **to access the records / data items** 2 Use of index to directly access a **record** in the array // Example of simplification of code e.g. use of dot notation Item[1].Stage 3 Simplifies the code / algorithm // Reduces duplication of code // **Program** easier to write / understand / maintain / test / debug // **Data items/record** easier to search / sort / manipulate	**3**

Examiner report missing
9618_s24_qp_22 Question 3

6(a)	Example solution:
	
	`Function Trim(Name : STRING) RETURNS STRING`
	`  CONSTANT Dots = "..."`
	`  CONSTANT Space = " "`
	
	`  IF LENGTH(Name) <= 16 THEN`
	`  RETURN Name`
	`  ENDIF`
	
	`  // Otherwise it has to be trimmed`
	
	`  WHILE LENGTH(Name) > 13`
	`  REPEAT`
	`  Name ← LEFT(Name, LENGTH(Name) - 1) // strip last char`
	`  UNTIL RIGHT(Name, 1) = Space // back to SPACE`
	`  ENDWHILE`
	
	`  Name ← LEFT(Name, LENGTH(Name) - 1) // remove the space`
	`  Name ← Name & Dots`
	`  RETURN Name`
	
	`ENDFUNCTION`
	
	Mark as follows:
	1. Function heading, ending, parameter and return type
	2. If length of original string <= 16 then return original string
	3. Any Conditional loop...
	4. until string is short enough
	5. Inner conditional loop / second condition to identify word(s) to remove from the end of string // Check to identify word(s) to remove from the end of string
	6. Attempt to strip characters back to space
	7. Correct removal of word/words from end of string **and** remove final space
	8. Concatenate `Dots` and return result
	
	Max 7 marks
		7

6(b)(i)	A (very) large **file** is created // redundant zeroes are stored in the file	**1**
6(b)(ii)	One mark for: • Values are delimited by a special character / a separator character • First character indicates **sample** length **Max 1 mark**	**1**
6(b)(iii)	The **algorithm** to **store / extract / separate** the individual values is more complex / takes longer to execute / run / process NE Algorithm is more complicated	**1**

Examiner report missing
9618_s24_qp_22 Question 6

8(a)	One mark for reason, one for benefit Reason: (Program is) easier to design / implement / test / debug / modify Benefit: Easier to check that **each stage** works as expected	**2**

8(b)	Example algorithm based on finding position of first non-space character and then using substring function:
	
	`FUNCTION DeleteSpaces(Line : STRING) RETURNS STRING`
	`  DECLARE NewLine : STRING`
	`  DECLARE EndOfLeading : BOOLEAN`
	`  DECLARE Count, NumSpaces : INTEGER`
	`  DECLARE NextChar : CHAR`
	`  CONSTANT Space = " "`
	
	`  NumSpaces ← 0`
	`  EndOfLeading ← FALSE`
	
	`  FOR Count ← 1 TO LENGTH(Line)`
	`  NextChar ← MID(Line, Count, 1)`
	`  IF NextChar <> Space AND EndOfLeading = FALSE THEN`
	`  NumSpaces ← Count - 1 // the number to trim`
	`  EndOfLeading = TRUE`
	`  ENDIF`
	`  NEXT Count`
	
	`  NewLine ← RIGHT(Line, LENGTH(Line) - NumSpaces)`
	
	`  RETURN NewLine`
	`ENDFUNCTION`
	
	Mark as follows:
	
	1 Loop to length of parameter `// Loop until first non-space character in Line`
	2 Extract a character **in a loop**
	3 Identify <u>first</u> non-space character **in a loop**
	4 Attempt at removing leading spaces in `Line`
	5 Leading spaces removed from `Line` / / Create new string without leading space
	6 Return a string following a reasonable attempt at removing leading spaces in `Line`
		6

8(c)	Example:
	
	`PROCEDURE Stage_2(F1, F2 : STRING)`
	
	`   DECLARE Line : STRING`
	`   DECLARE Count : INTEGER`
	
	`   Count ← 0`
	
	`   OPEN F1 FOR READ`
	`   OPEN F2 FOR APPEND`
	
	`   WHILE NOT EOF(F1)`
	`      READFILE F1, Line`
	`      Line ← DeleteSpaces(Line)`
	`      Line ← DeleteComment(Line)`
	`      IF Line <> "" THEN`
	`         WRITEFILE F2, Line  // skip blank lines`
	`      ELSE`
	`         Count ← Count + 1`
	`      ENDIF`
	`   ENDWHILE`
	
	`   CLOSEFILE F1`
	`   CLOSEFILE F2`
	
	`   OUTPUT Count, " blank lines were removed"`
	
	`ENDPROCEDURE`
	
	Mark as follows:
	
	1 Procedure heading, parameters, ending
	2 Open both files in correct modes **and** subsequently close
	3 Loop to `EOF(F1)`
	4 Read a line from `F1` **in a loop**
	5 Assign return values from `DeleteComment()` and `DeleteSpaces()` **in a loop**
	6 Check return value following both MP5 function calls is not an empty string and if so write to `F2` **in a loop**
	7 Count the blank lines **in a loop**
	8 Output number of blank lines removed following a reasonable attempt **after the loop**
		8

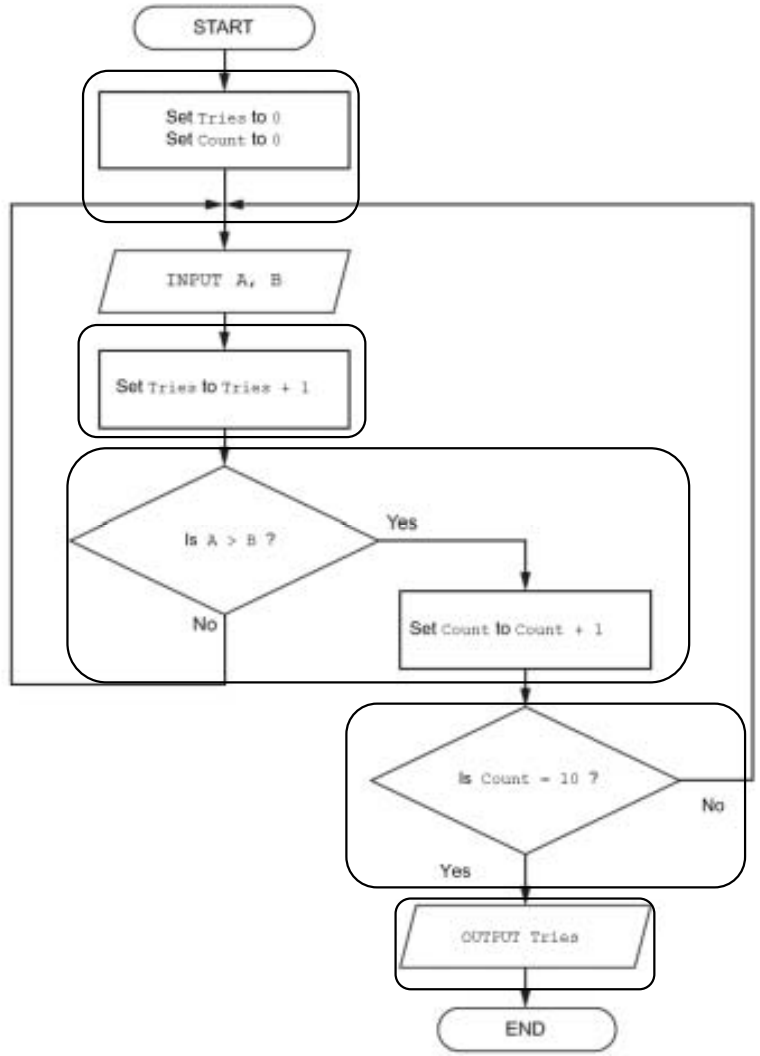
Examiner report missing
9618_s24_qp_22 Question 8

1(a)		**4**

Assignment statement	Data type
`A ← LEFT(MyName, 1)`	CHAR / STRING
`B ← Total * 2`	INTEGER / REAL
`C ← INT(ItemCost) / 3`	REAL
`D ← "Odd OR Even"`	STRING

One mark per row

Examiner report missing
9618_s24_qp_21 Question 1

2(a)	 One mark per outlined region: 1 Initialise both counts 2 Increment `Tries` every time a pair is input 3 Compare A > B **and** increment `Count` if TRUE 4 Test for Count = 10 (10th time A > B) – MUST include Yes / No labels 5 If so output `Tries`, otherwise loop	**5**

2(b)	One mark per point: 1 A (variable of type) string will be input // by example e.g. “67,72” 2 A special / identified character would need to be used to separate each numeric value // all numbers are fixed length	**2**

Examiner report missing
9618_s24_qp_21 Question 2

<table>
<tr><td>3(a)</td><td>One mark per point:

1 The PS contains a null pointer
2 The PF points to the first element on the free list
3 All the nodes are on the free list</td><td>3</td></tr>
<tr><td>3(b)</td><td>Max 2 marks for 'Variables':

<table><tr><td>The two variables will be of type Integer</td></tr><tr><td>The two variables will be used as pointers / indexes to the arrays.</td></tr><tr><td>The values stored in the two variables will indicate the first element in each list</td></tr><tr><td>The first 1D array will be of type String</td></tr><tr><td>The first 1D array will be used to store the values // data items // User IDs</td></tr><tr><td>The second 1D array will be of type Integer</td></tr><tr><td>The second 1D array will be used to store the pointers // point to next item</td></tr></table>
Mark as follows:

One mark for each of the first three rows
One mark for both Array 1 rows
One mark for both Array 2 rows</td><td>5</td></tr>
</table>

Examiner report missing
9618_s24_qp_21 Question 3

4	Example:
	
	`FUNCTION Check() RETURNS STRING`
	`  DECLARE Odd, Even, Index : INTEGER`
	
	`  Odd ← 0`
	`  Even ← 0`
	
	`  FOR Index ← 1 TO 100`
	`  IF Index MOD 2 = 0 THEN`
	`  Even ← Even + Data[Index]`
	`  ELSE`
	`  Odd ← Odd + Data[Index]`
	`  ENDIF`
	`  NEXT Index`
	
	`ENDFUNCTION`
	
	Mark as follows:
	1. Function heading, ending and return type
	2. Declare local variables `Odd`, `Even` and `Index` as integers
	3. Initialise `Odd` and `Even`
	4. Loop for 100 `iterations` // through array
	5. Sum `Odd` and `Even` element values **in a loop**
	6. Compare `Odd` and `Even` after the loop and Return appropriate string
		6

Examiner report missing
9618_s24_qp_21 Question 4

5(a)								**6**

Count	C	L	Index	Result	Data[1]	Data[2]	Data[3]
				"****"	"aaaaaa"	"bbbbbb"	"cccccc"
1	'X'	3	1				
				"AAAAAA"	"AAA"		
3	'Y'	2	2				
				"bbbbbb"		"bb"	
5	'W'	4	3				
							"bbbb"

One mark per zone.

Examiner report missing
9618_s24_qp_21 Question 5

8(a)	```
	FUNCTION DeleteComment(Line : STRING) RETURNS STRING
	   DECLARE NewLine, TwoChars : STRING
	   DECLARE Count, TrimTo : INTEGER
	   CONSTANT Comment = "//"
	
	   NewLine ← Line
	   TrimTo ← 0
	   Count ← 1
	   WHILE Count < LENGTH(Line) AND TrimTo = 0
	      TwoChars ← MID(Line, Count, 2) // extract 2 chars
	      IF TwoChars = Comment THEN
	         TrimTo ← Count
	      ENDIF
	      Count ← Count + 1
	   ENDWHILE
	
	   IF TrimTo <> 0 THEN
	      NewLine ← LEFT(Line, TrimTo - 1)
	   ENDIF
	
	   RETURN NewLine
	
	ENDFUNCTION
	``
	
	Mark as follows:
	
	1 Loop to length of `Line` (parameter) // length of `Line -1`
	2 Terminate loop on first double slash
	3 Attempt to extract one/two characters **in a loop**
	4 Check for "//" after attempt at extraction **in a loop**
	5 Record start position of comment // Calculate amount of trim required
	6 Attempt to trim `Line` from start of comment
	7 Completely correct trimmed `Line` from start of comment
	8 Return `Line` after a reasonable overall attempt
		8

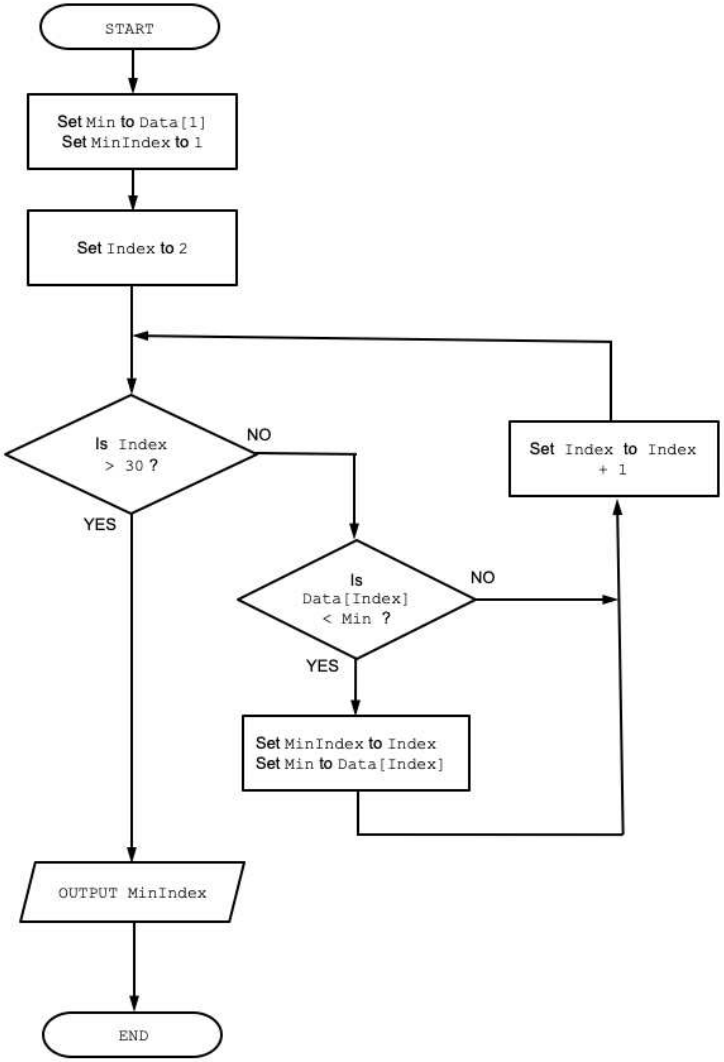
8(b)	Example:
	
	`FUNCTION Stage_1(StudentName : STRING) RETURNS INTEGER`
	
	`   DECLARE OldFile, NewFile, Line : STRING`
	`   DECLARE Count : INTEGER`
	
	`   OldFile ← StudentName & "_src.txt"`
	`   NewFile ← StudentName & "_S1.txt"`
	`   OPENFILE OldFile FOR READ`
	`   OPENFILE NewFile FOR WRITE`
	`   Count ← 0`
	
	`   WHILE NOT EOF(OldFile)`
	`      READFILE OldFile, Line`
	`      Line ← DeleteComment(Line)`
	`      IF LENGTH(Line) <> 0 THEN`
	`         WRITEFILE NewFile, Line`
	`         Count ← Count + 1`
	`      ENDIF`
	`   ENDWHILE`
	
	`   CLOSEFILE OldFile`
	`   CLOSEFILE NewFile`
	
	`   RETURN Count`
	`ENDFUNCTION`
	
	Mark as follows:
	
	1 Generate filenames condone missing "_"
	2 Open both files in correct modes **and** subsequently close
	3 Loop to `EOF(OldFile)`
	4 Read a line from `OldFile` **and** execute `DeleteComment()` **in a loop**
	5 Skip blank lines after `DeleteComment()` **in a loop**
	6 Write `Line` to stage 1 file **and** increment Count
	7 Return the number of lines after a reasonable attempt at counting
		7

Examiner report missing
9618_s24_qp_21 Question 8

1(a)		**4**

Assignment statement	Data type
MyVar1 ← Total1 / Total2	REAL
MyVar2 ← 27/10/2023	DATE
MyVar3 ← "Sum1 / Sum2"	STRING
MyVar4 ← Result1 AND Result2	BOOLEAN

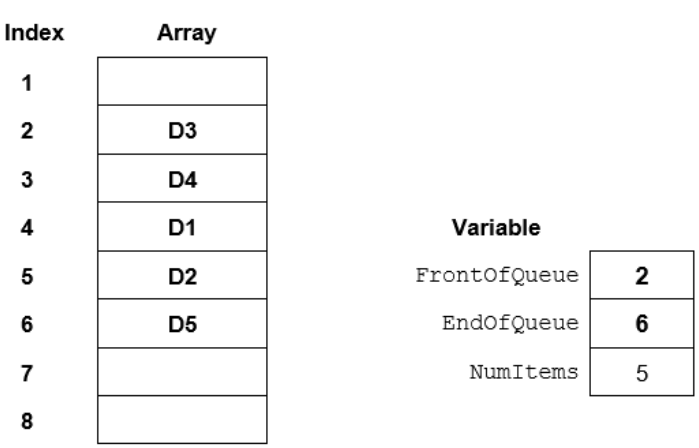
Examiner report missing
9618_w23_qp_23 Question 1

2(a)	 **MP1** Initialise `Min` to first value in `Data` **and** `MinIndex` to 1 **MP2** Loop through 29 more values (or 30 values in total) **MP3** Compare element from `Data[]` with `Min` **MP4** Set new `Min` **AND** save `MinIndex` when element value < `Min` **in a loop** **MP5** Output `MinIndex`	**5**

2(b)	**MP1** Simplifies the algorithm // easier to write / understand / test / debug **MP2** It is possible to iterate through the values // can use a loop // allows the storage of many values using a single identifier	**2**

2(c)	**One** mark per underlined section `DECLARE Data : ARRAY[1:120] OF REAL`	**2**

Examiner report missing
9618_w23_qp_23 Question 2

3(a)	 : **MP1** D3, D4, D1, D2 and D5 in question order or reversed - in any five consecutive locations **MP2** `FoQ` value matches index with D3 **MP3** `EoQ` value matches index with D5	**3**
3(b)	**MP1** If `NumItems` **is / = 8** // **(queue) full** then jump to step 6 **MP2** Increment **`EndOfQueue`** **MP3** If **`EndOfQueue = 9`** then set `EndOfQueue` to **1** **MP4** Increment **`NumItems`** **MP5** Set the **Element** at the index … **MP6** stored in **`EndOfQueue`** to **value/data/item** being added Mark as follows: Steps 1 to 4: One mark for gaps filled as shown Step 5: One mark for 'element' and one mark for other two terms	**6**

Examiner report missing
9618_w23_qp_23 Question 3

5		6

Index	Value	Total	Mix[1]	Mix[2]	Mix[3]	Mix[4]
2		0	4	2	3	5
	2	2				
				5		
3						
	3	5				
					8	
4						
	5	10				
						9
2						
	5	15				
				13		
2						
	13	28				
				21		
2						
			56			

MP1 Row 1 (initialisation)

Each iteration (1 – 5):
MP2 1 – Total 2
MP3 2 – Total 5
MP4 3 – Total 10
MP5 4 – Total 15
MP6 5 – Total 28 **and** final `Mix[1] =` 56

Examiner report missing
9618_w23_qp_23 Question 5

7(a)	**MP1** iteration / looping **MP2** naming all four modules correctly in the correct sequence // e.g. `Module-A` repeatedly calls `Sub-Y1`, then `SubY2` then `Sub-9`	**2**

7(b)(i)	`TYPE MyType`
	`  DECLARE RA  : INTEGER`
	`  DECLARE RB  : STRING`
	`  DECLARE RC  : BOOLEAN`
	`ENDTYPE`
	
	MP1 `TYPE MyType... ENDTYPE`
	MP2 `RA` as integer and `RB3` as string
	MP3 `RC` as Boolean
		3

Examiner report missing
9618_w23_qp_23 Question 7

8(a)	 PROCEDURE ReceiveFile(FileName : STRING) DECLARE FileData : STRING DECLARE CharCount : INTEGER CONSTANT Terminator = "****" OPENFILE FileName FOR WRITE FileData ← GetData() CharCount ← 0 WHILE FileData <> Terminator WRITEFILE FileName, FileData CharCount ← CharCount + LENGTH(FileData) FileData ← GetData() ENDWHILE CLOSEFILE FileName OUTPUT CharCount (, " characters were written to ", FileName) ENDPROCEDURE **MP1** OPEN file in WRITE mode and subsequently CLOSE **MP2** Conditional loop until terminator received **MP3** 'Attempted' use of `GetData()` – Ignore CALL … **MP4** Fully correct use `GetData()` to return the data **in a loop** **MP5** Maintain `CharCount` **in a loop** **MP6** Write each line to the file - **except** the terminator **in a loop** **MP7** Final output of message giving number of characters written **outside loop**	**7**

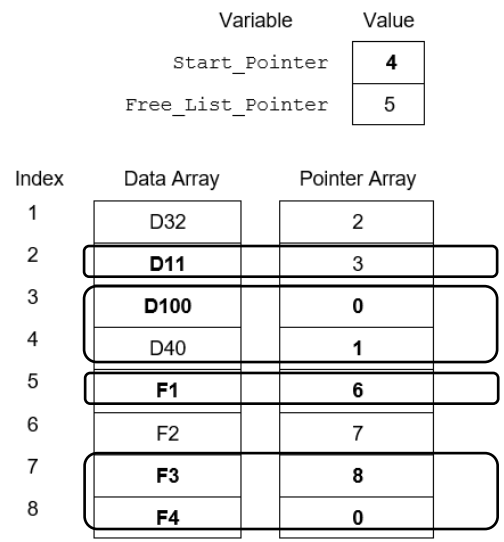
8(b)	**Max 3** marks Problem: • If the file being sent contains a line of the string "****" • then the file being written by `ReceiveFile()` will end at this point // subsequent file lines will be lost Solution: • Read the file (at the sending end) to find the number of lines it contains • Send an initial message which defines the number of lines in the file ALTERNATIVE SOLUTION: • (Transmitter program) chooses a **different** terminator string / character that doesn't occur in the file • Transmitter program sends the terminator string / character before first line of file / before the transfer begins	**3**

Examiner report missing
9618_w23_qp_23 Question 8

Examiner report missing
9618_w23_qp_23 Question 52

1(a)	**One** mark for each row with appropriate variable name and data type **Example value** / **Explanation** / **Variable name** / **Data type** `"Mr Khan"` / The name of the customer / **`CustomerName`** / **`STRING`** `3` / The number of items in the order / **`NumItems`** / **`INTEGER`** `TRUE` / A value to indicate whether this is a new customer / **`NewCustomer`** / **`BOOLEAN`** `15.75` / The deposit paid by the customer / **`Deposit`** / **`REAL`**	**4**
1(c)	**One** mark per point **Max 3** marks Declaration 1 Declare a composite / record (type) 2 Declare an array of the given composite / record (type) Expansion of record: 3 … containing all data items required // containing items of different data types Expansion of array: 4 … where **each array element** represents data for one order / customer (order)	**3**

Examiner report missing
9618_w23_qp_22 Question 1

3(a)	 Mark as follows: • One mark for `Start_Pointer` value • One mark each group of row(s): • 2 • 3 and 4 • 5 • 7 and 8 For null pointer: accept 0 / Ø / an out-of-bound index value less than 1, greater than 8	**5**

3(b)	One mark per step: 1 Assign the data item **D6** to **F1** 2 Set the **pointer** of this node to point to **D11** 3 Set Ptr2 to point to **F2** 4 Set pointer of **D32** to point to **D6**	**4**

Examiner report missing
9618_w23_qp_22 Question 3

5		6

Index	Value	Count	Mix[1]	Mix[2]	Mix[3]	Mix[4]
3		0	1	3	4	2
	4				3	
4		1				
	2					1
2		2				
	3			2		
3		3				
	3				2	
3		4				
	2				1	
2		5				
						10

One mark for:

- Initialisation row
- Each iteration of Count 1 to 4 with correct `Index`, `Count` and array `Mix` as shown
- Final iteration, Count = 5, correct `Index`, `Count` and array `Mix` as shown plus `Mix[4]` assignment

Examiner report missing
9618_w23_qp_22 Question 5

6(a)	Example Solution
	
	`PROCEDURE CreateFiles(NameRoot : STRING, NumFiles : INTEGER)`
	`  DECLARE FileName, Suffix : STRING`
	`  DECLARE Count : INTEGER`
	
	`  FOR Count ← 1 TO NumFiles`
	
	`  Suffix ← NUM_TO_STR(Count)`
	`  WHILE LENGTH(Suffix) <> 3`
	`  Suffix ← '0' & Suffix`
	`  ENDWHILE`
	
	`  FileName ← NameRoot & '.' & Suffix`
	`  OPENFILE FileName FOR WRITE`
	`  WRITEFILE FileName, "This is File " & FileName`
	`  CLOSEFILE FileName`
	
	`  NEXT Count`
	
	`ENDPROCEDURE`
	
	Mark as follows:
	1 Procedure heading, including parameters, and ending
	2 Loop for `NumFiles` iterations
	3 Attempt to create **filename suffix** using `NUM_TO_STR()` **in a loop**
	4 Completely correct filename
	5 `OPENFILE` in `WRITE` mode and subsequent `CLOSE` **in a loop**
	6 `WRITE` initial first line to the file **in a loop**
		6

6(b)(i)	Function	**1**

6(b)(iii)	Read	**1**

Examiner report missing
9618_w23_qp_22 Question 6

8(a)	Solution using a loop
	
	Example solution – using a loop
	
	`FUNCTION GetPort(ThisDest : STRING) RETURNS INTEGER`
	`  DECLARE Index, DNum, Port : INTEGER`
	
	`  DNum ← STR_TO_NUM(ThisDest)`
	`  Index ← 1`
	`  Port ← -1`
	
	`  REPEAT`
	`  IF RouteTable[Index, 1] <> -1 THEN`
	`  IF DNum >= RouteTable[Index, 1] AND __`
	`  DNum <= RouteTable[Index, 2] THEN`
	`  Port ← RouteTable[Index, 3]`
	`  ENDIF`
	`  ENDIF`
	`  Index ← Index + 1`
	`  UNTIL Index = 7 OR Port <> -1  // end of array or range found`
	
	`  RETURN Port`
	
	`ENDFUNCTION`
	
	Mark as follows **Max 7**:
	1.Function heading and ending **including** parameter and return type
	2 Convert **parameter** to a number
	3 (Conditional) loop through array
	4 Skip unused element **in a loop**
	5 Attempt to check one range with destination **in a loop**
	6 Test **all** ranges correctly with destination **in a loop**
	7 Store port value if destination matched **in a loop** (and exit loop)
	8 Return port value including -1 if no match found
	
	Solution using selection statement(s)
	
	Example solution
	
	`FUNCTION GetPort(ThisDest : STRING) RETURNS INTEGER`
	`  DECLARE Index, DNum, Port : INTEGER`
	
	`  DNum ← STR_TO_NUM(ThisDest)`
	`  Port ← -1`
	
	`  IF RouteTable[1, 1] <> -1 AND RouteTable[1, 1] >= DNum AND RouteTable[1, 2] <= DNum THEN`
		7

8(b)	Example solution
	
	`PROCEDURE ProcessMsg(ThisMsg : STRING)`
	`  DECLARE ThisDest : STRING`
	`  DECLARE Response : BOOLEAN`
	`  DECLARE StackNum : INTEGER`
	
	`  IF LENGTH(ThisMsg) >= 4 THEN`
	
	`  ThisDest ← LEFT(ThisMsg, 3)`
	
	`  IF ThisDest = MyID THEN  // It's for this computer`
	`  StackNum ← 1`
	`  ELSE`
	`  StackNum ← 2`
	`  ENDIF`
	
	`  Response ← StackMsg(ThisMsg, StackNum)`
	
	`  IF Response =  FALSE THEN`
	`  OUTPUT "Message discarded - no room on stack"`
	`  ENDIF`
	
	`  ENDIF`
	`ENDPROCEDURE`
	
	Mark as follows:
	1. Ignore message if data field is empty
	2. Extract `ThisDest` from `ThisMsg`
	3. Test if destination is this computer
	4. Attempt to use `StackMsg()`
	5. **Fully correct** use of `StackMsg()` for **both** cases / stacks
	6. Test `StackMsg()` return value for **both** cases
	7. Following a reasonable attempt at MP6 output warning if `StackMsg()` returns `FALSE`
		7

8(c)(i)	One mark per point **Max 3** marks: **Decide on scenario and mark accordingly.** **Scenario one**: • If more than one line is / all lines are stored on the stack (before line(s) are removed) • The stack operates as a FILO device // Last item added to stack will be in first item out • So **lines in the file** appear out of sequence **Scenario two:** • Stack is Full • Not all lines can be stored on the stack • so resulting **file** will not contain **all the original lines** **Scenario three:** • (All) the data in a line read can't be stored on the stack • Stack elements have not been allocated enough memory • so only **part of each line** is stored in the **file** **Scenario four:** • Stack is empty • The stack is being read faster than it is being written to • so **blank lines** may be inserted into the **file**	**3**

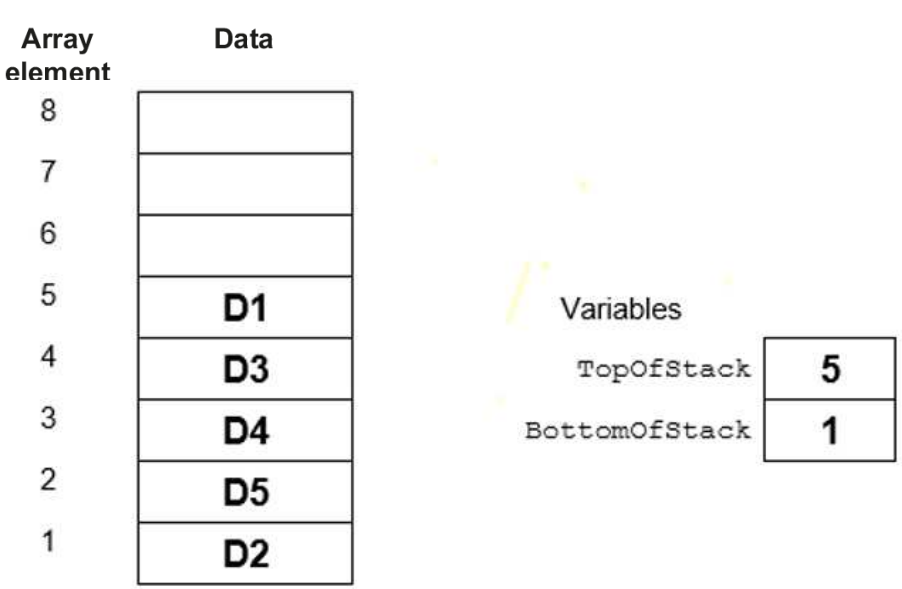
8(c)(ii)	Queue	**1**

Examiner report missing
9618_w23_qp_22 Question 8

1(a)	**One** mark per row: |	Answer
---	---
The value assigned to `Level` when `ThisValue` is 40	**"Medium"**
The value assigned to Check when `ThisValue` is 36	**12**
The value assigned to Level when `ThisValue` is 18	**"Low"**
The number of elements in array Data that may be incremented	**11**		**4**

1(d)	**One** mark per point: • `ThisValue`: **INTEGER** • `Check`: **REAL** • `Level`: **STRING**	**3**

Examiner report missing
9618_w23_qp_21 Question 1

3(a)	 **MP1** all values in the order and location shown **MP2** `TopOfStack` value is index of element containing D1 **MP3** `BottomOfStack` value is index of element containing D2	**3**
3(b)	**MP1** If **`TopOfStack = 8`** // **(stack) full** then return `FALSE` **MP2** Otherwise, **increment** `TopOfStack` **MP3** Use `TopOfStack` as an **index** to the Array **MP4** Set the element at this **index** / **location** / **position** to the **value** / **data** / **item** being added **MP5** Return **`TRUE`**	**5**

Examiner report missing
9618_w23_qp_21 Question 3

4(a)	 FUNCTION TooMany(Search : STRING, Max : INTEGER) RETURNS BOOLEAN DECLARE Count, Index : INTEGER Count ← 0 FOR Index ← 1 TO 150 IF Data[Index] = Search THEN Count ← Count + 1 ENDIF NEXT Index IF Count > Max THEN RETURN TRUE ELSE RETURN FALSE ENDIF ENDFUNCTION **MP1** Function heading, ending and return type **MP2** Declare Count and Index as integers **MP3** Initialise Count **MP4** Loop (any type) for 150 iterations **MP5** Compare Data element with parameter - if equal, increment Count in a loop **MP6** Compare Count with Max and return Boolean in both cases outside the loop	6

4(b)	**MP1** Test for row being even number
	MP2 Test for either column value equal to `Search`
	
	`IF Row MOD 2 = 0 AND __`
	
	`(Data[Row, 1] = Search OR Data[Row, 2] = Search) THEN`
	
	ALTERNATIVE using nested `IFs`:
	
	`IF Row MOD 2 = 0 THEN`
	
	`IF Data[Row, 1] = Search OR Data[Row, 2] = Search THEN`
	
	MP3 Selection structure is either:
	• Single IF statement using AND, or
	• Two nested IFs using AND, or
	• Single IF and the use of a two-iteration loop
	Either of these structures correctly formed scores the mark
	
	ALTERNATIVE SOLUTION: A FOR loop using 'STEP 2'
	
	`FOR Row ← 2 TO 150 / NEXT Row STEP 2`
	
	`Data[Row, 1] = Search OR Data[Row, 2] = Search) THEN`
		3

Examiner report missing
9618_w23_qp_21 Question 4

8(a)	``` PROCEDURE SendFile(FileName, DestID : STRING, Port : INTEGER) DECLARE FileData : STRING CONSTANT STX = CHR(02) CONSTANT ETX = CHR(03) OPENFILE FileName FOR READ WHILE NOT EOF(FileName) READFILE FileName, FileData FileData ← STX & DestID & MyID & FileData & ETX CALL Transmit(FileData, Port) ENDWHILE CLOSEFILE FileName CALL Transmit(STX & DestID & MyID & "****" & ETX, Port) ENDPROCEDURE ``` Mark as follows: **MP1** OPEN file in READ mode – using parameter - and subsequently CLOSE **MP2** Conditional loop to `EOF()` **MP3** Use of `READFILE` to get a line from the file **MP4** 'Attempt' to form a message (minimum is `DestID, MyID, FileData`) **MP5** Message formed is completely correct **MP6** Call `Transmit()` with correct MP4 string **in a loop** **MP7** Transmit the "****" message (all parts present) **after the loop**	**7**
8(b)	**Max 2** marks **MP1** Indicates that all the lines of the file have been sent // it is the end of the transmission / file transfer **MP2** So that the receiving program can stop waiting for further data **MP3** The file can be closed / saved	**2**

8(c)(i)	**MP1** A message cannot contain a zero-length data field **MP2** … so a blank line cannot be sent // there is no way to send a blank line	**2**

8(c)(ii)	**MP1** Append a (special) character to the start of the message text **MP2** interpret the new field data as a blank line ALTERNATIVE **MP1** Change the message protocol and use an additional field to act as an indicator **MP2** Interpret the new field data	**2**

Examiner report missing
9618_w23_qp_21 Question 8

1(a)	**Answer** The dimension of the array: **2** The name of the variable used as an array index: **PCount** The number of elements in the array: **100**	**3**
1(c)	Integer	**1**

Examiner report missing
9618_s23_qp_23 Question 1

3(a)(i)	6	**1**

3(a)(ii)	**Stack** **Pointer** **Memory location** / **Value** 506 / 505 / BBB 504 / AAA 503 / XXX ← TopOfStack 502 / ZZZ 501 / NNN 500 / PPP ← BottomOfStack **Variable** / **Value** Data1 / WWW Data2 / AAA One mark for: MP1 Values 'BBB' and 'AAA' MP2 Values 'XXX' to 'PPP' (unchanged) MP3 Both pointers and labelled22 MP4 Values of both variables	**4**

3(b)(i)	So that the data may be recovered / restored (the next time the program is run) // the data is permanently saved **/** data is not lost when the program terminates	**1**

3(b)(ii)	**Max 5 marks** MP1 Open the text file in WRITE mode MP2 Check there is a value on the stack MP3 POP value …. MP4 Write value to the text file MP5 Repeat from Step 2 // loop referencing the stack items Alternative solution: Not using POP primitive MP1 Open the text file in WRITE mode MP2 Check there is a value on the stack MP3 Read value from ToS location MP4 Write the value to the text file – Must some attempt at 'the value' NOT '<u>all</u> the values' MP5 Decrement ToS MP6 Repeat from step 2 // loop referencing the stack items	**5**

Examiner report missing
9618_s23_qp_23 Question 3

8(a)		**8**

```
FUNCTION ChangeSupp(Code1, Code2 : STRING) RETURNS
                                                    INTEGER
  DECLARE Count : INTEGER
  DECLARE ThisLine, ThisCode : STRING

  OPENFILE "Stock.txt" FOR READ
  OPENFILE "NewStock.txt" FOR WRITE
  Count ← 0
  WHILE NOT EOF("Stock.txt")
     READFILE("Stock.txt ", ThisLine) // brackets
                                                    optional
     ThisCode ← MID(ThisLine, 5, 3)
     IF ThisCode = Code1 THEN
        ThisLine ← LEFT(ThisLine, 4) & Code2
                        & RIGHT(ThisLine,
                                  LENGTH(ThisLine) - 7)
        Count ← Count + 1
     ENDIF
     WRITEFILE("NewStock.txt", ThisLine) // brackets
                                                    optional
  ENDWHILE

  CLOSEFILE "NewStock.txt"
  CLOSEFILE "Stock.txt"

RETURN Count
ENDFUNCTION
```

MP1 Open both files, in correct modes, and subsequently close
MP2 Conditional loop until `EOF("Stock.txt")`
MP3 Read a line from `Stock.txt` **AND** extract `ThisCode` **in a loop**
MP4 Test `ThisCode = Code1` AND if true, increment `Count` (must have been Initialised) **in a loop**
MP5 Update `ThisLine` using substring functions and '&' **in a loop**
MP6 completely correct update of `ThisLine` **in a loop**
MP7 Write `ThisLine` to `NewStock.txt` **in a loop**
MP8 Return count after loop

8(b)		**6**

```
PROCEDURE Report_1(Supp : STRING)
  DECLARE Count : INTEGER
  DECLARE ThisItemNum, ThisDesc, ThisLine, ThisCode :
                                                    STRING

  Count ← 0

  OPENFILE "Stock.txt" FOR READ

  OUTPUT "Report for Supplier:" & Supp
  OUTPUT ""           //Blank line as per example
  OUTPUT "Item        Description"
  OUTPUT ""           //Blank line as per example

  WHILE NOT EOF("Stock.txt")
     READFILE("Stock.txt", ThisLine)
     ThisCode ← Mid(ThisLine, 5, 3)
     IF ThisCode = Supp THEN
        ThisItemNum ← LEFT(ThisLine, 4)
        ThisDesc ← RIGHT(ThisLine, LENGTH(ThisLine) - 7)
        OUTPUT ThisItem & "        " & ThisDesc
        Count ← Count + 1
     ENDIF
  ENDWHILE

  CLOSEFILE "Stock.txt"

  OUTPUT "" //Blank line as per example
  OUTPUT "Number of items listed: ", Count
ENDPROCEDURE
```

MP1 Output report header (blank lines optional) – Must contain the parameter code
MP2 Conditional loop until `EOF("Stock.txt")`
MP3 Read a line from `Stock.txt` **AND** extract `SupplierCode` **in a loop**
MP4 Test if `SupplierCode = Supp` then `increment count` (must have been Initialised)
MP5 Extract **AND** output item and description **in a loop**
MP6 Output the final line with count

8(c)(i)	**Max 2 marks** MP1 Must ‘calculate’ the count before any item + description output / after the file is read once MP2 Lines to be output have to be stored … MP3 The file has to be read twice	**2**

8(c)(ii)	One mark per point: MP1 Loop through the file calculating the count MP2 Save ‘selected’ items in an array MP3 (After all lines have been read), output the header lines / count MP4 Loop through the array to output each array element	**3**

Examiner report missing
9618_s23_qp_23 Question 8

1(a)(i)	Use of constants	**1**

1(c)	One mark per bullet point: • BOOLEAN • REAL • STRING	**3**

Examiner report missing
9618_s23_qp_22 Question 1

2(a)	`MyDOB ← SETDATE(17, 11, 2007)`	**1**

2(c)	One mark per array definition bullet: • A (1D) array containing 7 elements • of type STRING One mark per Step: Step1: Assign value "Sunday" to first element, "Monday" to second element etc. Step2: Use the function `DAYINDEX()` to return / find the day number from `MyDoB` Step3: Use the returned value as the array index / to access the element that contains the name / string Step4: Output the element / name / string **Note: Max 2 for Array definition, Max 4 for steps**	**6**

Examiner report missing
9618_s23_qp_22 Question 2

3(a)(i)	One mark per point: 1 Check that the queue is not full 2 `EoQ` pointer will move to point to location 9 3 Data item Orange will be stored in location referenced by EoQ pointer 4 `EoQ` pointer will move to point to location 0 5 Data item Yellow will be stored in location referenced by EoQ pointer **Note: max 4 marks**	**4**

3(a)(ii)	7	**1**

3(b)	One mark per bullet: 1 Open file in `READ` mode 2 Loop to `EOF()` // read / process all the lines in file 3 Loop will end when return value from `AddToQueue()` is `FALSE` / queue is full 4 Read a line from the file in a loop 5 Pass string to `AddToQueue()` // `AddToQueue()` is executed with line as parameter	**5**

Examiner report missing
9618_s23_qp_22 Question 3

8(a)	 FUNCTION CheckInfo(NewLine: STRING) RETURNS BOOLEAN DECLARE ThisNum : STRING DECLARE Index : INTEGER IF LENGTH(NewLine) < 19 THEN RETURN FALSE ENDIF FOR Index ← 1 TO 4 IF NOT IS_NUM(MID(NewLine, Index, 1)) THEN RETURN FALSE ENDIF NEXT Index ThisNum ← LEFT(Newline, 4) IF ThisNum < "0001" OR ThisNum > "5999" THEN RETURN FALSE ENDIF IF NOT OnlyAlpha(MID(Newline, 5, 3)) THEN RETURN FALSE ENDIF RETURN TRUE ENDFUNCTION Mark as follows: 1 Test length of parameter 2 Extract first 4 characters of parameter (as ItemNum) 3 Test first four characters are **all** numeric 4 Test ItemNum in range "0001" to "5999" 5 Extract characters 5 to 7 of parameter (as SupplierCode) 6 Use of OnlyAlpha() with extracted SupplierCode 7 Return BOOLEAN value correctly in all cases, must have been declared as local	**7**

8(b)		**7**

```
PROCEDURE AddItem(NewLine : STRING)
  DECLARE NewItemNum, ThisItemNum : STRING

  OPENFILE "Stock.txt" FOR READ
  OPENFILE "NewStock.txt" FOR WRITE
  NewItemNum ← LEFT(NewLine, 4)

  WHILE NOT EOF("Stock.txt")
     READFILE("Stock.txt", ThisLine)
     ThisItemNum ← LEFT(ThisLine, 4)
     IF ThisItemNum > NewItemNum THEN
        WRITEFILE("NewStock.txt", NewLine) // write New
                                              Line...
        NewItemNum ← "9999" // ...once only
     ENDIF
     WRITEFILE("NewStock.txt", ThisLine)
  ENDWHILE

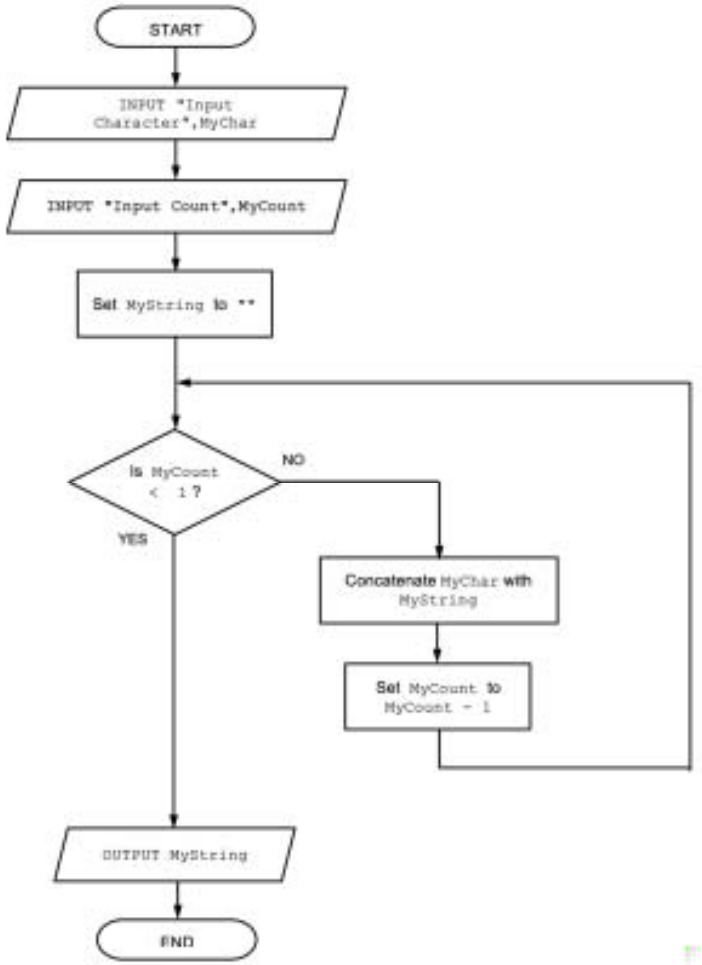
  IF NewItemNum <> "9999" THEN
     WRITEFILE("NewStock.txt", NewLine) //New last line
                                          in the file
  ENDIF

  CLOSEFILE "Stock.txt"
  CLOSEFILE "NewStock.txt"
ENDPROCEDURE
```

Mark as follows:

1 Open both files, in correct modes, and subsequently close
2 Conditional loop until end of file `Stock.txt`
3 Read a line from `Stock.txt` **AND** extract `ThisItemNum` **in a loop**
4 Test `ThisItemNum > NewItemNum` then write `NewLine` to `NewStock.txt`
5 ...including mechanism to only do this **once only**
6 Write line read from `Stock` to `NewStock`
7 Deal with the case where `NewLine` is the new last line

Examiner report missing
9618_s23_qp_22 Question 8

2(a)	 One mark per point: 1 Both prompts 2 Both inputs using correct identifiers as given in question, `MyChar` and `MyCount` 3 Initialise `MyString` to empty string **and** subsequent output **after loop** 4 Loop `MyCount` iterations including decrement `MyCount` 5 Use of Concatenate or equivalent pseudocode statement **inside loop** **Max 4 Marks**	**4**

2(b)	One mark for declaration: DECLARE StartDate : DATE One mark for each underlined part of assignment: StartDate ← SETDATE (15, 11, 2005)	**3**

Examiner report missing
9618_s23_qp_21 Question 2

3(a)(i)	One mark for structure Structure: Record One mark for each point Advantage: • A set of data / all data related to one customer • of different types • is held under a single identifier/entity	**4**

3(a)(ii)	A (1D) array of records // An array of the given type could be used One mark per underlined word	**2**

Examiner report missing
9618_s23_qp_21 Question 3

6	 PROCEDURE Mix() DECLARE Count, Total ThisNum : INTEGER DECLARE ThisUser, ThisSample : INTEGER FOR ThisSample ← 1 TO 128 Count ← 0 Total ← 0 FOR ThisUser ← 1 TO 6 IF Sample[ThisUser, ThisSample] > 10 THEN Count ← Count + 1 Total ← Total + Sample[ThisUser, ThisSample] ENDIF NEXT ThisUser Result[ThisSample] ← INT(Total / Count) NEXT ThisSample ENDPROCEDURE Mark as follows: 1 Declaration **and** initialisation before inner loop of `Count` and `Total` 2 Outer Loop for 128 iterations 3 Inner loop for six iterations 4 Test for sample > 10 **in a loop** 5 and if true sum `Total` and increment `Count` 6 Calculate average value and assign to `Result` array **after inner loop and within outer loop** 7 Use of `INT()/ DIV` to convert average to integer **Max 6 Marks**	**6**

Examiner report missing
9618_s23_qp_21 Question 6

8(a)	 ``` FUNCTION IsNewSupp(ThisString : STRING) RETURNS BOOLEAN DECLARE Index : INTEGER DECLARE ThisChar : CHAR IF LENGTH(ThisString) <> 5 THEN RETURN FALSE // invalid SupplierCode length ENDIF IF SuppExists(ThisString) THEN RETURN FALSE // SupplierCode already exists ENDIF FOR Index ← 1 TO 5 ThisChar ← TO_LOWER(MID(ThisString, Index, 1)) IF ThisChar < 'a' OR ThisChar > 'z'THEN RETURN FALSE ENDIF NEXT Index RETURN TRUE ENDFUNCTION ``` Mark as follows: 1 Check `ThisString` is exactly 5 characters in length 2 Use of `SuppExists()` with a string of 5 characters as a parameter 3 Loop for 5 iterations // loops for length of string parameter 4 Extract a character **in a loop** 5 Test if char is alphabetic **in a loop** 6 **…**catering for both upper and lower case 7 Return Boolean following a reasonable attempt	**7**

8(b)	
	``
	FUNCTION CheckNewItem(NewLine : STRING) RETURNS BOOLEAN
	  DECLARE NotFound : BOOLEAN
	  DECLARE NewItemNum, ThisItemNum, ThisLine : STRING
	
	  NotFound ← TRUE
	
	  OPENFILE "Stock.txt" FOR READ
	  NewItemNum ← LEFT(NewLine, 4)
	  ThisItemNum ← "0000" //rogue initial value
	
	  WHILE NOT EOF("Stock.txt") AND NotFound = TRUE AND__
	                                ThisItemNum < NewItemNum
	    READFILE("Stock.txt", ThisLine) //brackets optional
	    ThisItemNum ← LEFT(ThisLine, 4)
	    IF ThisItemNum = NewItemNum THEN
	      NotFound ← FALSE
	    ENDIF
	  ENDWHILE
	
	  CLOSEFILE "Stock.txt"
	
	  RETURN NotFound
	ENDFUNCTION
	``
	
	Mark as follows:
	
	1 Open `Stock.txt` in READ mode and subsequently close
	2 Extract `NewItemNum` from parameter
	3 Conditional loop until `EOF("Stock.txt")`
	4 ... **OR** `NewItemNum` found
	5 ... **OR** when `ThisItemNum < NewItemNum`
	6 Read a line from `Stock.txt` **AND** extract `ThisItemNum` **in a loop**
	7 If `ThisItemNum = NewItemNum` then terminate loop / set flag **in a loop**
	8 Return Boolean after reasonable attempt
	
	Max 7 marks
	
	Max 6 if function wrapper (heading and ending) missing or incorrect
		7

8(d)	Append	**1**

8(e)	One mark for each part: • The algorithm / search / iteration can stop /only iterates • if the current value read from the file // current line in file • is greater than the value being searched for	**3**

Examiner report missing
9618_s23_qp_21 Question 8

1(a)	One mark per row **Variable use** / **Data type** Store the number of days in the current month — **INTEGER** Store the first letter of the customer's first name — **CHAR** Store an indication of whether a year is a leap year — **BOOLEAN** Store the average amount spent per customer visit — **REAL**	**4**

Examiner report missing
9618_w22_qp_23 Question 1

4(a)	The data type (of the item to be stored)	**1**

4(b)(i)	Operation: Add an item / Enqueue Check: There are unused elements in the array // The queue is not full Operation: Remove an item / Dequeue Check: There are items in the array // The queue is not empty One mark for reason and one mark for reason it could not be completed.	**4**

4(b)(ii)	One mark per point (**Max 5**): 1 Declare a (1D) array of size >= 10 2 …of data type `CHAR` 3 Declare integer variable for `FrontOfQueuePointer` 4 Declare integer variable for `EndOfQueuePointer` 5 Initialise `FrontOfQueuePointer` and `EndOfQueuePointer` to represent an empty queue 6 Declare integer variable or `NumberInQueue` 7 Declare integer variable for `SizeOfQueue` to count / limit the max number of items allowed // Reference to mechanism for defining 'wrap' of circular queue 8 Initialise `SizeOfQueue` // Initialise `NumberInQueue`	**5**

Examiner report missing
9618_w22_qp_23 Question 4

5(a)(i)	One mark per point:
	
	1 Procedure heading and ending including four parameters...
	2 ...and use of `BYREF` for the three extracted values
	3 Extract and assign `SID`
	4 Extract and assign `SDesc`
	5 Calculation of length of `SCost` (remainder of string)
	6 Extract and assign `SCost` following an attempt at MP5
	
	`PROCEDURE UnPack(BYVAL TLine : STRING, BYREF SID, SDesc, SCost : STRING)`
	`  SID ← LEFT(TLine, 5)`
	`  SDesc ← MID(TLine, 6, 32)`
	`  SCost ← RIGHT(TLine, LENGTH(TLine) - 37)`
	`ENDPROCEDURE`
		6

5(b)(i)	`LineData.Cost ← 12.99`	**1**

Examiner report missing
9618_w22_qp_23 Question 5

7(a)	One mark per point (**Max 8**):
	
	1 Declaration and initialisation of local integer for `Count`
	2 Appropriate prompt and two inputs
	3 (Conditional) loop while error number input is in range // error code 999 reached
	4 …and not end of array
	5 Check if this `ErrCode` needs to be output **in a loop**
	6 if so check for blank error text **in a loop**
	7 Output in both cases
	8 ….and increment count **in a loop**
	9 OUTPUT of header and summary including count
	
	`PROCEDURE OutputRange()`
	`  DECLARE First, Last, Count, Index, ThisErr : INTEGER`
	`  DECLARE ThisMess : STRING`
	`  DECLARE PastLast: BOOLEAN`
	
	`  Count ← 0`
	`  Index ← 1`
	`  PastLast ← FALSE`
	
	`  OUTPUT "Please input first error number: "`
	`  INPUT First`
	`  OUTPUT "Please input last error number: "`
	`  INPUT Last`
	
	`  OUTPUT "List of error numbers from ", First, " to ", Last`
	
	`  WHILE Index < 501 AND NOT PastLast`
	`  ThisErr ← ErrCode[Index]`
	`  IF ThisErr > Last THEN`
	`  PastLast ← TRUE`
	`  ELSE`
	`  IF ThisErr >= First THEN`
	`  ThisMess ← ErrText[Index]`
	`  IF ThisMess = "" THEN`
	`  ThisMess ← "Error Text Missing"`
	`  ENDIF`
	`  OUTPUT ThisErr, " : ", ThisMess`
	`  Count ← Count + 1`
	`  ENDIF`
	`  ENDIF`
	`  Index ← Index + 1`
	`  ENDWHILE`
	
	`  OUTPUT Count, " error numbers output"`
	
	`ENDPROCEDURE`
		8

7(b)(i)	One mark per point:
	
	1 (Conditional) loop terminating when item added **OR** end of array reached
	2 Test for unused element **in a loop**
	3 Assignment of values to arrays // save index of first blank location and assign after loop
	4 Set loop termination if empty element found in a loop
	5 Call `SortArrays()` **once**
	6 Calculation of remaining unused elements **and** return Integer value (for both cases)
	
	`FUNCTION AddError(ErrNum : INTEGER, ErrMess : STRING)`
	`RETURNS INTEGER`
	`  DECLARE Index, Remaining : INTEGER`
	`  CONSTANT Unused = 999`
	
	`  Index ← 1`
	`  Remaining ← -1`
	
	`  REPEAT`
	`    IF ErrCode[Index] = Unused THEN`
	`      ErrCode[Index] ← ErrNum`
	`      ErrText[Index] ← ErrMess`
	`      CALL SortArrays()`
	`      Remaining ← 500 - Index`
	`    ENDIF`
	`    Index ← Index + 1`
	`  UNTIL Remaining <> -1 OR Index > 500`
	
	`  RETURN Remaining`
	
	`ENDFUNCTION`
		6

7(b)(ii)	One mark per point (**Max 3**): 1. Loop through 500 elements (while error number not found) 2. Compare `ErrCode` for current element with the error number 3. If same, set element value to 999 (and terminate loop) 4. ... and call `SortArrays()` (to move 999 to the end) – once only	**3**

Examiner report missing
9618_w22_qp_23 Question 7

2(a)(i)	One mark per step (or equivalent): 1 Open file in APPEND mode (and subsequent Close) 2 Prompt and Input a student name and mark 3 If mark greater than or equal to 20 jump to step 5 4 Write only the name to the file 5 Repeat from Step 2 for 35 times / the number of students	**5**

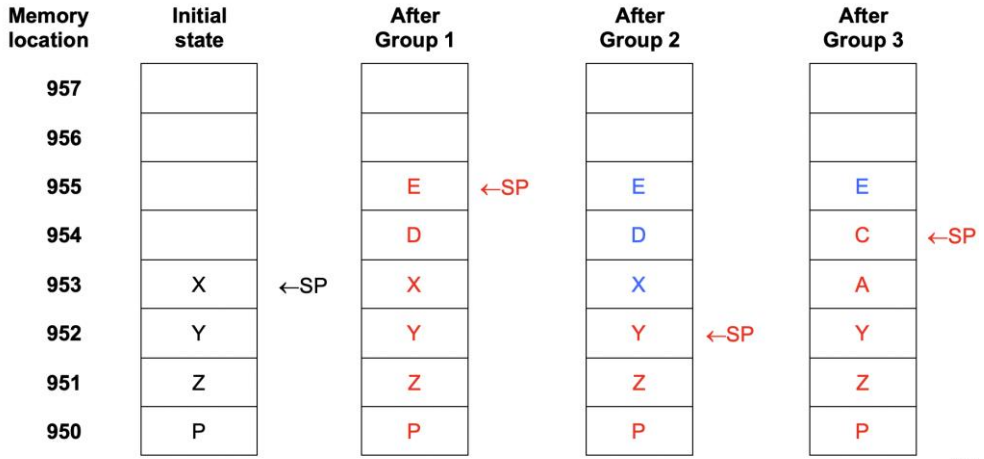
2(a)(ii)	Data in a file is saved after the computer is switched off / stored permanently // no need to re-enter the data when the program is re-run	**1**

2(a)(iii)	Example answer: So that existing file data is not overwritten.	**1**

Examiner report missing
9618_w22_qp_22 Question 2

3(a)	One mark for name, Max two for features (**Max 3 in total**) Name: Queue Features: 1 Each queue element contains one data item 2 A Pointer to the front / start of the queue 3 A Pointer to the back / end of the queue 4 Data is added at back / end and removed from front / start // works on a FIFO basis 5 May be circular **ALTERNATIVE:** Name: Linked List Features: 1 Each node contains data and a pointer to the next node 2 A Pointer to the start of the list 3 Last node in the list has a null pointer 4 Data may be added / removed by manipulating pointers (not moving data) 5 Nodes are traversed in a specific sequence 6 Unused nodes are stored on a free list // a free-list pointer to the Free List	**3**

3(b)	One mark per point (**Max 5**): 1 Declare a (1D) array of data type `STRING` 2 The number of elements in that array corresponds to the size of the required stack 3 Declare an integer / variable for `StackPointer` 4. Declare an integer / variable for the size of the stack // for the max value of `StackPointer` 5 Use the `StackPointer` as an index to the array 6 Pointers and variables initialised to indicate empty stack 7 Store each item on the stack as one array element / Each stack item maps to one array element 8 Attempt to describe Push **and** Pop operations 9 Push **and** Pop routines need to check for full or empty conditions	**5**

3(c)	One mark for each: 1 Data 'After Group 1' (as shown, including blank cells) 2 Data 'After Group 2' (as shown, including blank cells) 3 Data 'After Group 3' (as shown, including blank cells) 4 SP 'After Group 1' pointing to location 955 5 Final two SPs pointing to locations 952 and 954  [5]	**5**

Examiner report missing
9618_w22_qp_22 Question 3

7(a)	One mark per point (**Max 6**):
	
	1 Procedure heading and ending **including** parameters
	2 Conditional loop containing incrementing `Index`...
	3 ...terminating when `ErrNum` found
	4 ...terminating when `ErrCode[Index] > ErrNum` (i.e. `ErrNum` not found)
	5 ... **OR** after element 500 tested
	6 Test if found and OUTPUT 'Found' message
	7 ...otherwise OUTPUT 'Not Found' message
	
	`PROCEDURE OutputError(LineNum, ErrNum : INTEGER)`
	`  DECLARE Index : INTEGER`
	
	`  Index ← 0`
	
	`  // Search until ErrNum found OR not present OR end of array`
	
	`  REPEAT`
	`  Index ← Index + 1`
	`  UNTIL ErrCode[Index] >= ErrNum OR Index = 500`
	
	`  IF ErrCode[Index] = ErrNum THEN`
	`  OUTPUT ErrText[Index], " on line ", LineNum //Found`
	`  ELSE`
	`  OUTPUT "Unknown error on line ", LineNum //Not found`
	`  ENDIF`
	
	`ENDPROCEURE`
		6

7(b)	One mark per point (**Max 8**): 1 Procedure heading and ending as shown 2 Conditional loop correctly terminated 3 An inner loop 4 Correct range for inner loop 5 Comparison (element J with J+1) **in a loop** 6 Swap elements in **both** arrays **in a loop** 7 'No-Swap' mechanism: • Conditional **outer** loop including flag reset • Flag set in **inner** loop to indicate swap 8 Efficiency (this scenario): terminate inner loop when `ErrCode = 999` 9 Reducing `Boundary` **in the <u>outer</u> loop**	**8**

```
PROCEDURE SortArrays()
  DECLARE TempInt, J, Boundary : INTEGER
  DECLARE TempStr : STRING
  DECLARE NoSwaps : BOOLEAN

  Boundary ← 499

  REPEAT
     NoSwaps ← TRUE
     FOR J ← 1 TO Boundary
        IF ErrCode[J]> ErrCode[J+1] THEN
           //first swap ErrCode elements
           TempInt ← ErrCode[J]
           ErrCode[J] ← ErrCode[J+1]
           ErrCode[J+1] ← TempInt
           //now swap corresponding ErrText elements
           TempStr ← ErrText[J]
           ErrText[J] ← ErrText[J+1]
           ErrText[J+1] ← TempStr
           NoSwaps ← FALSE
        ENDIF
     NEXT J
     Boundary ← Boundary - 1
  UNTIL NoSwaps = TRUE

ENDPROCEDURE
```

7(c)(i)	`ErrCode` should be an `INTEGER` // `ErrCode` should not be a `STRING`	**1**

7(c)(ii)	Benefits include: 1 Array of records can store mixed data types / multiple data types under a single identifer 2 Tighter / closer association between `ErrCode` and `ErrText` // simpler code as fields may be referenced together // values cannot get out of step as with two arrays 3 Program easier to design / write / debug / test / maintain / understand One mark per point **Note: Max 2 marks**	**2**

7(c)(iii)	`DECLARE Error : ARRAY[1:500] OF ErrorRec`	**1**

Examiner report missing
9618_w22_qp_22 Question 7

1(a)	One mark per point: 1 They are tried and tested so free from errors 2 They perform a function that you may not be able to program yourself (for example **encryption**) 3 They are readily available / speed up development time	**3**

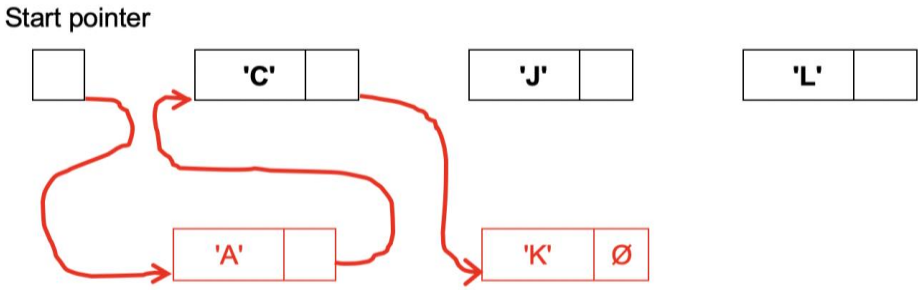
1(b)(i)	One mark per point: 1 Algorithm to process / search / organise the data is easier to implement // Values may be accessed via a loop-controlled variable / iterated through using index 2 Makes the program easier to design / code / test / understand 3 Multiple instances referenced via a single identifier / so fewer identifiers needed // Easier to amend the program when the number of students increases	**3**

1(b)(ii)	One mark per point: **Purpose**: It identifies / references an individual array element // provides the index to the array **Data type**: Integer	**2**

Examiner report missing
9618_w22_qp_21 Question 1

3	One mark per point (**Max 5**): 1 Declare a variable / an integer `Max` 2 Assign value of first / any element to `Max` 3 Set up a loop to repeat 200 times / from start to end of array 4 Use the loop counter as the array index 5 If value of current element is greater than `Max`... 6 ...then assign value to `Max` 7 After the loop, Output `Max`	**5**

Examiner report missing
9618_w22_qp_21 Question 3

4(a)(i)	One mark for each: 1 Data A and K stored in new / existing nodes 2 Start pointer points to Node A 3 Node A points to Node C **and** Node C points to Node K 4 Node K contains Null Pointer 	**4**
4(a)(ii)	One mark per point: 1 Pointers determine the ordering of data // only the pointers need to be changed when data changed 2 Easier to add / delete data (to maintain correct sequence) in a linked list // description of moving data to maintain correct sequence when array used	**2**
4(a)(iii)	One mark per point: 1 Need to store pointers as well as data 2 More complex (to setup / implement)	**2**

4(b)	One mark per point (**Max 4**): 1 Declare **two** (1D) arrays 2 One for data, one for pointers 3 Elements from same index represent one node 4 Declare an integer / variable for `StartPointer` // explain its use 5 Define appropriate value for null pointer // explain its use 6 Declare an integer / variable for `FreeList` pointer // explain its use 7 Routines are needed to add / delete / search Alternative MP1, 2 and 3 for record-based implementation: 1 Define a record type with fields for data and pointer 2 Declare **one** (1D) array 3 ...of the defined record type	**4**

Examiner report missing
9618_w22_qp_21 Question 4

5	Mark as follows (**Max 5**):
	
	1 Procedure heading and ending **and** declaration of both indexes
	2 Loop to process all elements from `Array1`
	3 Sum (any) three consecutive values from `Array1` **and** divide by 3 **in a loop**
	4 Convert result to Integer
	5 Assign value to correct element of `Array2` **in a loop**
	6 Increment `Array2` index **in a loop**
	
	`PROCEDURE Summarise()`
	`  DECLARE Value : REAL`
	`  DECLARE IxA, IxB : INTEGER // Index variables`
	
	`  IxB ← 1`
	
	`  FOR IxA ← 1 TO 598 STEP 3`
	`  Value ← Array1[IxA] + Array1[IxA + 1] + Array1[IxA + 2]`
	`  Value ← Value / 3`
	`  Array2[IxB] ← INT(Value)`
	`  IxB ← IxB + 1`
	`  NEXT IxA`
	
	`ENDPROCEDURE`
		5

Examiner report missing
9618_w22_qp_21 Question 5

8(a)	One mark for each point (**Max 7**) as follows: 1 Function heading and ending **including** parameter and return type 2 Declaration **and** initialisation of local Integer for `Count` 3 OPEN in READ mode and CLOSE 4 Conditional loop until `EOF()` 5 Read a line **in a loop** 6 If non-blank, increment count **in a loop** 7 Terminate loop when 10 non-blank lines have been read 8 Return Boolean in both cases	**7**

```
FUNCTION CheckFile(Thisfile : STRING) RETURNS BOOLEAN
   DECLARE Valid : BOOLEAN
   DECLARE ThisLine : STRING
   DECLARE Count : INTEGER

   Count ← 0
   Valid ← FALSE
   OPEN ThisFile FOR READ

   WHILE NOT EOF(ThisFile) AND Valid = FALSE
      READFILE ThisFile, ThisLine
      IF ThisLine <> "" THEN
         Count ← Count + 1
         IF Count > 9 THEN
            Valid ← TRUE
         ENDIF
      ENDIF
   ENDWHILE

   CLOSEFILE ThisFile
   RETURN Valid

ENDFUNCTION
```

8(b)	`CALL CountErrors("Jim01Prog.txt", 20)` One mark for each: 1 Module name, at least one parameter in brackets and one parameter correct 2 Completely correct statement	**2**

8(c)	Mark as follows:
	1 Procedure heading and ending **including** parameters
	2 Declaration and initialisation of local Integer value for `ErrCount`
	3 Use of `CheckFile()`, output message **and** terminate if it returns `FALSE`
	4 Conditional loop until `EOF()`
	5 ...or `ErrCount > MaxErrors`
	6 Read line and use as parameter to `CheckLine()` **in a loop**
	7 Test return value and increment `ErrCount` if non-zero **in a loop**
	8 Output either message once only as appropriate
	
	`PROCEDURE CountErrors(ThisFile : STRING, MaxErrors : INTEGER)`
	`  DECLARE ErrCount, ThisError : INTEGER`
	`  DECLARE ThisLine : STRING`
	
	`  ErrCount ← 0`
	
	`  IF CheckFile(ThisFile) = FALSE THEN`
	`  OUTPUT "That program file is not valid"`
	`  ELSE`
	`  OPEN ThisFile FOR READ`
	
	`  REPEAT`
	`  READFILE, ThisFile, ThisLine`
	`  ThisError ← CheckLine(ThisLine)`
	`  IF ThisError <> 0 THEN`
	`  ErrCount ← ErrCount + 1`
	`  ENDIF`
	`  UNTIL ErrCount > MaxErrors OR EOF(ThisFile)`
	
	`  IF EOF(ThisFile) = FALSE THEN`
	`  OUTPUT "Check terminated - too many errors"`
	`  ELSE`
	`  OUTPUT "There were ", ErrCount, " errors."`
	`  ENDIF`
	
	`  CLOSEFILE ThisFile`
	`  ENDIF`
	
	`ENDPROCEDURE`
		8

Examiner report missing
9618_w22_qp_21 Question 8

1(a)(i)	One from: • The program obeys the rules / grammar of the programming language used • The program will run // it can be compiled / interpreted • Accept by example. e.g. 'no mis-spelt keywords' / 'all brackets match'	**1**

1(b)	<table><tr><th>Use of variable</th><th>Data type</th></tr><tr><td>The average mark in a class of 40 students</td><td>REAL</td></tr><tr><td>An email address</td><td>STRING</td></tr><tr><td>The number of students in the class</td><td>INTEGER</td></tr><tr><td>Indicate whether an email has been read</td><td>BOOLEAN</td></tr></table> One mark per row	**4**

Examiner report missing
9618_w21_qp_23 Question 1

2(a)	One mark for reference to: 1 The use a variable as an index to the array 2 A loop to iterate through the array 3 An Inner loop (with a reducing range) 4 Test if current element is greater than next element 5 if so then swap elements 6 Description of swap 7 Attempt at efficient algorithm **Max 6** marks	**6**

Examiner report missing
9618_w21_qp_23 Question 2

3(a)(i)	One mark per point: • `EoQ` pointer will move to point to location 4 // incremented `EoQ` (by 1) • Data value "Octopus" will be stored in location pointed to be `EoQ` / location 4	**2**

3(a)(ii)	One mark for each bullet • Value "Frog" // value pointed to by `FoQ` / location 0 is assigned to variable `AnimalName` • `FoQ` pointer will move to point to location 1 / point to "Cat" // incremented `FoQ` (by 1) **0** Frog ← **Front of queue pointer** **1** Cat **2** Fish **3** Elk ← **End of queue pointer**	**2**

3(a)(iii)	There is only one data item in the queue	**1**

3(b)(i)	One mark for data values plus one mark for pointers **0** Frog **1** Cat **2** Fish ← **Front of queue pointer** **3** Elk **4** **Wasp** **5** **Bee** **6** **Mouse** ← **End of queue pointer** **7** One mark for each pointer One mark for three new data values	**3**

3(b)(ii)	<table><tr><td>0</td><td>Shark</td><td>← End of queue pointer</td></tr><tr><td>1</td><td>(Cat)</td><td></td></tr><tr><td>2</td><td>(Fish)</td><td></td></tr><tr><td>3</td><td>(Elk)</td><td></td></tr><tr><td>4</td><td>Wasp</td><td>← Front of queue pointer</td></tr><tr><td>5</td><td>Bee</td><td></td></tr><tr><td>6</td><td>Mouse</td><td></td></tr><tr><td>7</td><td>Dolphin</td><td></td></tr></table> One mark for BOTH pointers One mark for all data values as shown	**2**
3(c)	One mark per point: 1 If incremented `EoQ` = `FoQ` then error condition: queue is full 2 Increment the `EoQ` 3 Manage wrap-around	**3**

Examiner report missing
9618_w21_qp_23 Question 3

5(a)	One mark for the character and one for the corresponding reason. • Character: Any except alphabetic, numeric, ',' ':' or space • Reason: character doesn't occur in data to be recorded	**2**

5(c)		**7**

```
FUNCTION LogEvents(StudentID : STRING) RETURNS INTEGER

  DECLARE FileData : STRING
  DECLARE Index, Count : INTEGER

  CONSTANT LogFile = "LogFile"

  Count ← 0
  OPENFILE LogFile FOR APPEND
  FOR Index ← 1 TO 2000
     FileData ← LogArray[Index]

     IF LEFT(FileData, 6) = StudentID THEN
        WRITEFILE (LogFile, FileData) //brackets optional
        Count ← Count + 1
        LogArray[Index] ← ""        // clear the element
     ENDIF
  NEXT Index

  CLOSEFILE LogFile

  RETURN Count

ENDFUNCTION
```

1 mark for each of the following:

1 Function heading and ending including parameter **and** return type
2 `OPEN` file `LogFile` for `APPEND` and subsequent `CLOSE`
3 Loop for 2000 iterations
4 Extract first 6 characters from array element **in a loop**
5 Compare first 6 characters with parameter **in a loop**
6 If equal:
- write whole array element string to file **and**
- increment `Count` **and**
- clear array element **in a loop**

7 Return Count (must have been declared **and** initialised)

Examiner report missing
9618_w21_qp_23 Question 5

6(a)	```
	PROCEDURE SetRow(Row, SkipNum, SetNum : INTEGER)
	  DECLARE Col : INTEGER
	
	  // array is 1280 x 800
	
	  FOR Col ← SkipNum + 1 TO SkipNum + SetNum
	    Screen[Row, Col] ← 1
	  NEXT Index
	
	ENDPROCEDURE
	``
	ALTERNATIVE 1:
	``
	  FOR Col ← 1 TO SetNum
	    Screen[Row, SkipNum + Col] ← 1
	  NEXT Col
	``
	ALTERNATIVE 2:
	``
	  WHILE SetNum > 0
	    Screen[Row, SkipNum + SetNum] ← 1
	    SetNum ← SetNum - 1
	  ENDWHILE
	``
	Mark as follows:
	
	1 Procedure heading and ending including parameters
	2 Declaration of local Integer for `Col`
	3 Count-controlled loop with meaningful start number
	4 correct stop number
	5 Reference Screen Array element and set to 1 **in a loop**
		5

6(b)	```
	FUNCTION SearchInRow(ThisRow, StartCol : INTEGER) RETURNS
	        INTEGER
	   DECLARE ThisCol, Step : INTEGER
	   DECLARE Found: BOOLEAN
	
	   // array is 1280 x 800
	
	   Found ← FALSE
	   ThisCol ← StartCol
	
	   // first decide which way to search
	   IF StartCol = 1 THEN
	      Step ← 1
	      EndCol ← 1281
	   ELSE
	       Step ← -1
	       EndCol ← 0
	   ENDIF
	
	   WHILE ThisCol <> EndCol AND Found = FALSE
	      IF Screen[ThisRow, ThisCol] <> 1 THEN
	         ThisCol ← ThisCol + Step
	      ELSE
	         Found ← TRUE
	      ENDIF
	   ENDWHILE
	
	   IF Found = FALSE THEN
	      ThisCol ← -1
	   ENDIF
	
	   RETURN ThisCol
	
	ENDFUNCTION
	``
	
	Mark as follows:
	
	1 Interpreting `StartCol` parameter to determine direction of search
	2 An attempt at searching both up and down
	3 Conditional Loop / Count-controlled loop with use of `ThisCol` index
	4 Using correct values for `StartCol`, `EndCol` and `Step`
	5 Reference a `Screen` element and compare with 1 **in a loop**
	6 If equal save column or immediately Return column **in a loop**
	7 Return column number or −1
	Loop(s) terminate when element with value = 1 found
	
	Max 7 marks if function heading, including return type, and ending is incorrect or incomplete
		8

Examiner report missing
9618_w21_qp_23 Question 6

1(a)	The process involves: 1 Breaking down a problem / task into sub problems / steps / smaller parts 2 In order to explain / understand // easier to solve the problem 3 Leading to the concept of program modules // assigning problem parts to teams **Max 2**	**2**

1(b)		**Answer**	**3**
	The number of dimensions of `ThisArray`	**1**	
	The technical terms for minimum and maximum values that variable `n` may take	**Lower bound, upper bound**	
	The technical term for the variable `n` in the pseudocode expression.	**Index / Subscript**	
	One mark per row		

Examiner report missing
9618_w21_qp_22 Question 1

3(a)(i)	Pseudocode:
	
	`TYPE Student`
	`  DECLARE StudentID : STRING`
	`  DECLARE Email : STRING`
	`  DECLARE Club_1 : INTEGER`
	`  DECLARE Club_2 : INTEGER`
	`  DECLARE Club_3 : INTEGER`
	`ENDTYPE`
	
	Mark as follows:
	
	• One mark for `TYPE` and `ENDTYPE`
	• One mark for `StudentID` **and** `Email` fields as `STRING`
	• One mark for all Club fields as `INTEGER`
		3

3(a)(ii)	`DECLARE Membership : ARRAY [1:3000] OF Student` One mark per underlined phrase	**2**
3(a)(iii)	One mark for any one of: • Assign a value (of the corrrect data type) outside the normal range to one of the fields • Assign an empty string to the `StudentID` field / `Email` field • or value out of range to any club field	**1**
3(a)(iv)	A number outside the range 1 to 99	**1**

3(b)	
	`PROCEDURE GetIDs()`
	`  DECLARE Index : INTEGER`
	`  DECLARE ThisClub, Count : INTEGER`
	
	`  OUTPUT "Please Input Club Number: "`
	`  INPUT ThisClub`
	
	`  Count ← 0`
	
	`  FOR Index ← 1 TO 3000`
	`  IF Membership[Index].Club_1 = ThisClub OR __`
	`  Membership[Index].Club_2 = ThisClub OR __`
	`  Membership[Index].Club_3 = ThisClub THEN`
	`  Count ← Count + 1`
	`  OUTPUT Membership[Index].StudentID`
	`  ENDIF`
	
	`  NEXT Index`
	
	`  OUTPUT "There are ", Count, " Students in the club"`
	`ENDPROCEDURE`
	
	Mark as follows:
	
	1 Declare and initialise `Count`
	2 Prompt and Input club number **before the loop**
	3 Loop through 3000 elements
	4 Compare **one** club field with number input
	5 Compare **all** `Club` fields with number input
	6 If number found, `OUTPUT` of `StudentID` field **and** increment `Count`
	7 Final `OUTPUT` of `Count` **outside the loop**
	
	Note:
	
	Max 6 if procedure heading and ending missing or incorrect (but allow array as parameter)
		7

Examiner report missing
9618_w21_qp_22 Question 3

5(a)	 ``` PROCEDURE MakeNewFile(OldFile, NewFile, Status : STRING) DECLARE Line1, Line2, Line3 : STRING DECLARE NumCopied, NumRecs : INTEGER NumRecs ← 0 NumCopied ← 0 OPENFILE OldFile FOR READ OPENFILE NewFile FOR WRITE WHILE NOT EOF(OldFile) READFILE OldFile, Line1 READFILE OldFile, Line2 READFILE OldFile, Line3 NumRecs ← NumRecs + 1 IF Line3 <> Status THEN WRITEFILE NewFile, Line1 WRITEFILE NewFile, Line2 WRITEFILE NewFile, Line3 NumCopied ← NumCopied + 1 ENDIF ENDWHILE OUTPUT "File " , OldFile , " contained " , NumRecs ,__ " employee details" OUTPUT Numcopied , " employee sets of details were __ written to file", NewFile CLOSEFILE OldFile CLOSEFILE NewFile ENDPROCEDURE ``` Mark as follows: 1 Procedure heading and ending, including parameters 2 `OPEN OldFile` for `READ` and `NewFile` for `WRITE` and subsequently `CLOSE` both files 3 Conditional loop until `EOF(OldFile)` 4 Read three lines from `OldFile` **in a loop** 5 Compare 3rd line read with `Status` parameter **and** if not equal write 3 lines to `NewFile` **in a loop** 6 Count number of sets read and those written **in a loop** 7 Final output including both counts and file names with suitable text **after a loop** Note: MP6: Both counts must have been declared and initialised	**7**

5(b)(i)	Store all three items on one line	**1**

5(b)(ii)	One mark per point: Advantage: Fewer file operations required Disadvantage: Algorithm to combine / extract individual data items is more complex	**2**

Examiner report missing
9618_w21_qp_22 Question 5

6(a)	```
	FUNCTION FirstRowSet() RETURNS INTEGER
	  DECLARE Row, Col : INTEGER
	  DECLARE Found : BOOLEAN
	
	  // array is 1280 × 800
	  Row ← 1
	
	  Found ← FALSE
	  WHILE Row <= 800 AND Found = FALSE // top to bottom
	  Col ← 1
	  WHILE Col <= 1280 AND Found = FALSE // left to right
	  IF Screen[Row,Col] = 1 THEN
	  Found ← TRUE // end function as soon as first
	  // found
	  ENDIF
	  Col ← Col + 1
	  ENDWHILE
	  Row ← Row + 1
	  ENDWHILE
	
	  IF Found = FALSE THEN // nothing found
	  Row ← 0
	  ENDIF
	  RETURN Row - 1
	ENDFUNCTION
	``
	
	Mark as follows:
	
	1 Function heading and ending **and** return type
	2 (Conditional) outer loop 1 to 800 (row)
	3 (Conditional) inner loop 1 to 1280 // 1280 to 1 (column)
	4 Reference `Screen` element and test for = 1 // <> 0
	5 and if true save row number **and** exit loops
	6 Increment index variables in both inner **and** outer loop
	7 Return Row number or −1, following a reasonable attempt
		7

6(b)	One mark for: • (A flag is used to) exit the loops // iteration is terminated • as soon as a Screen element with value 1 is found	**2**

Examiner report missing
9618_w21_qp_22 Question 6

<table>
<tr><td>1(a)(i)</td><td>One from:

• The program obeys the rules / grammar of the programming language used
• The program will run // it can be compiled / interpreted
• Accept by example. e.g. 'no mis-spelt keywords' / 'all brackets match'</td><td>1</td></tr>
<tr><td>1(b)</td><td><table><tr><th>Use of variable</th><th>Data type</th></tr><tr><td>The average mark in a class of 40 students</td><td>REAL</td></tr><tr><td>An email address</td><td>STRING</td></tr><tr><td>The number of students in the class</td><td>INTEGER</td></tr><tr><td>Indicate whether an email has been read</td><td>BOOLEAN</td></tr></table>
One mark per row</td><td>4</td></tr>
</table>

Examiner report missing
9618_w21_qp_21 Question 1

3(a)(i)	One mark per point: • `EoQ` pointer will move to point to location 4 // incremented `EoQ` (by 1) • Data value "Octopus" will be stored in location pointed to be `EoQ` / location 4	**2**

3(a)(ii)	One mark for each bullet • Value "Frog" // value pointed to by `FoQ` / location 0 is assigned to variable `AnimalName` • `FoQ` pointer will move to point to location 1 / point to "Cat" // incremented `FoQ` (by 1) **0** Frog ← **Front of queue pointer** **1** Cat **2** Fish **3** Elk ← **End of queue pointer**	**2**

3(a)(iii)	There is only one data item in the queue	**1**

3(b)(i)	One mark for data values plus one mark for pointers **0** Frog **1** Cat **2** Fish ← **Front of queue pointer** **3** Elk **4** **Wasp** **5** **Bee** **6** **Mouse** ← **End of queue pointer** **7** One mark for each pointer One mark for three new data values	**3**

3(b)(ii)	<table><tr><td>**0**</td><td>**Shark**</td><td>← **End of queue pointer**</td></tr><tr><td>**1**</td><td>(Cat)</td><td></td></tr><tr><td>**2**</td><td>(Fish)</td><td></td></tr><tr><td>**3**</td><td>(Elk)</td><td></td></tr><tr><td>**4**</td><td>Wasp</td><td>← **Front of queue pointer**</td></tr><tr><td>**5**</td><td>Bee</td><td></td></tr><tr><td>**6**</td><td>Mouse</td><td></td></tr><tr><td>**7**</td><td>**Dolphin**</td><td></td></tr></table> One mark for BOTH pointers One mark for all data values as shown	**2**
3(c)	One mark per point: 1 If incremented `EoQ` = `FoQ` then error condition: queue is full 2 Increment the `EoQ` 3 Manage wrap-around	**3**

Examiner report missing
9618_w21_qp_21 Question 3

5(a)	One mark for the character and one for the corresponding reason. • Character: Any except alphabetic, numeric, ',' ':' or space • Reason: character doesn't occur in data to be recorded	**2**

5(c)		**7**

```
FUNCTION LogEvents(StudentID : STRING) RETURNS INTEGER

  DECLARE FileData : STRING
  DECLARE Index, Count : INTEGER

  CONSTANT LogFile = "LogFile"

  Count ← 0
  OPENFILE LogFile FOR APPEND
  FOR Index ← 1 TO 2000
     FileData ← LogArray[Index]

     IF LEFT(FileData, 6) = StudentID THEN
        WRITEFILE (LogFile, FileData) //brackets optional
        Count ← Count + 1
        LogArray[Index] ← ""        // clear the element
     ENDIF
  NEXT Index

  CLOSEFILE LogFile

  RETURN Count

ENDFUNCTION
```

1 mark for each of the following:

1 Function heading and ending including parameter **and** return type
2 `OPEN` file `LogFile` for `APPEND` and subsequent `CLOSE`
3 Loop for 2000 iterations
4 Extract first 6 characters from array element **in a loop**
5 Compare first 6 characters with parameter **in a loop**
6 If equal:
- write whole array element string to file **and**
- increment `Count` **and**
- clear array element **in a loop**

7 Return Count (must have been declared **and** initialised)

Examiner report missing
9618_w21_qp_21 Question 5

6(a)	```
	PROCEDURE SetRow(Row, SkipNum, SetNum : INTEGER)
	  DECLARE Col : INTEGER
	
	  // array is 1280 x 800
	
	  FOR Col ← SkipNum + 1 TO SkipNum + SetNum
	    Screen[Row, Col] ← 1
	  NEXT Index
	
	ENDPROCEDURE
	``
	ALTERNATIVE 1:
	``
	  FOR Col ← 1 TO SetNum
	    Screen[Row, SkipNum + Col] ← 1
	  NEXT Col
	``
	ALTERNATIVE 2:
	``
	  WHILE SetNum > 0
	    Screen[Row, SkipNum + SetNum] ← 1
	    SetNum ← SetNum - 1
	  ENDWHILE
	``
	Mark as follows:
	
	1 Procedure heading and ending including parameters
	2 Declaration of local Integer for `Col`
	3 Count-controlled loop with meaningful start number
	4 correct stop number
	5 Reference Screen Array element and set to 1 **in a loop**
		5

6(b)	```
	FUNCTION SearchInRow(ThisRow, StartCol : INTEGER) RETURNS
	        INTEGER
	   DECLARE ThisCol, Step : INTEGER
	   DECLARE Found: BOOLEAN
	
	   // array is 1280 x 800
	
	   Found ← FALSE
	   ThisCol ← StartCol
	
	   // first decide which way to search
	   IF StartCol = 1 THEN
	      Step ← 1
	      EndCol ← 1281
	   ELSE
	       Step ← -1
	       EndCol ← 0
	   ENDIF
	
	   WHILE ThisCol <> EndCol AND Found = FALSE
	      IF Screen[ThisRow, ThisCol] <> 1 THEN
	         ThisCol ← ThisCol + Step
	      ELSE
	         Found ← TRUE
	      ENDIF
	   ENDWHILE
	
	   IF Found = FALSE THEN
	      ThisCol ← -1
	   ENDIF
	
	   RETURN ThisCol
	
	ENDFUNCTION
	``
	
	Mark as follows:
	
	1 Interpreting `StartCol` parameter to determine direction of search
	2 An attempt at searching both up and down
	3 Conditional Loop / Count-controlled loop with use of `ThisCol` index
	4 Using correct values for `StartCol`, `EndCol` and `Step`
	5 Reference a `Screen` element and compare with 1 **in a loop**
	6 If equal save column or immediately Return column **in a loop**
	7 Return column number or −1
	Loop(s) terminate when element with value = 1 found
	
	Max 7 marks if function heading, including return type, and ending is incorrect or incomplete
		8

Examiner report missing
9618_w21_qp_21 Question 6

1(a)		**4**

Example value	Explanation	Variable name	Data type
`"Wong"`	The preferred name of the member joining the football club	**`MemberName`**	**`STRING`**
`FALSE`	A value to indicate whether an existing member of the club lives at the same address	**`FamilyMember`**	**`BOOLEAN`**
`19/02/1983`	When the member joined the football club	**`StartDate`**	**`DATE`**
`1345`	The number of points a member has earned. Members of the club earn points for different activities.	**`Points`**	**`INTEGER`**

One mark for each appropriate variable name plus data type

1(c)	Structure: • Record Justification: • Allows different data types • to be stored under one identifier Mark as follows: One mark for Structure Two marks for justification. **Alternative** • Two (1D) arrays • One of string, one of integer • Where same index links the name with the score	**3**

Examiner report missing
9618_s21_qp_23 Question 1

3(a)(i)	One mark per bullet point: • Data from the arrays is written to the files at the end of the day / before the program is terminated / computer is switched off • Data can then be read from the files at the start of the next day and written to / stored in the arrays • No need to (re-)enter the data manually // only need to enter data once **Note: Max 2 marks**	**2**

3(a)(ii)	• The data is retained when the program is terminated / after the computer is switched off // data is stored permanently // non-volatile storage	**1**

3(a)(iii)	One mark per bullet point: • Data items are combined to form a single string / saved as a single line in the file • Data items are separated by a special character // make each data item a fixed length ALTERNATIVE: • Convert all data items / 'number of people' to strings • Consecutive / each line stores a separate data item	**2**

3(b)	``` PROCEDURE Preview (ThisFile : STRING) DECLARE LineNum : INTEGER DECLARE ThisLine : STRING OPENFILE ThisFile FOR READ IF EOF(ThisFile) THEN OUTPUT "Warning Message" ELSE LineNum ← 1 WHILE LineNum < 6 AND NOT EOF(ThisFile) READFILE Thisfile, ThisLine OUTPUT ThisLine LineNum ← LineNum + 1 ENDWHILE ENDIF CLOSEFILE ThisFile ENDPROCEDURE ``` Marks as follows: 1 Procedure heading (including parameter) and ending 2 File OPEN and subsequently CLOSE 3 Check if file is empty **and** output a warning message if it is 4 Conditional Loop 5 Output line (including blank lines) and read next line **in a loop**	**5**

Examiner report missing
9618_s21_qp_23 Question 3

5	``` PROCEDURE Sort() DECLARE Temp : INTEGER DECLARE NoSwaps : BOOLEAN DECLARE Boundary, Row, Col : INTEGER Boundary ← 999 REPEAT NoSwaps ← TRUE FOR Row ← 1 TO Boundary IF Result[Row, 2] > Result[Row + 1, 2] THEN FOR Col ← 1 TO 2 Temp ← Result [Row, Col] Result [Row, Col] ← Result [Row + 1, Col] Result [Row + 1, Col] ← Temp NEXT Col NoSwaps ← FALSE ENDIF NEXT J Boundary ← Boundary - 1 UNTIL NoSwaps = TRUE ENDPROCEDURE ``` Mark as follows: 1 Outer loop 2 Inner loop 3 Correct comparison **in a loop** 4 Correct swap of col1 array elements **in a loop** 5 Correct swap of col2 array elements **in a loop** (via loop or separate statements) 6 `'NoSwap'` mechanism: Conditional outer loop including flag reset 7 `'NoSwap'` mechanism: Set flag in inner loop to indicate swap 8 Reducing Boundary **in the <u>outer</u> loop**	**8**

Examiner report missing
9618_s21_qp_23 Question 5

6(a)	One mark per point: 1 Check for a free node 2 Search for correct insertion point 3 Assign data value B to first node in free list / node pointed to by start pointer of free list 4 Pointer from A will be changed to point to node containing B (instead of C) 5 Pointer from B will be changed to point to node containing C 6 Start pointer in free list moved to point to next free node **Note: max 4 marks**	**4**

6(b)	One mark per point: • An array (1D) to store the data **and** a second array (1D) to store the pointers • An (integer) variable to hold the start pointer **and** an (integer) variable to store the next free pointer ALTERNATIVE: • Define a record type comprising a data element and a pointer **and** declare an array (1D) of this type • An integer variable to hold the start pointer **and** an integer variable to store the next free pointer	**2**

Examiner report missing
9618_s21_qp_23 Question 6

7(a)	
	``
	FUNCTION GetStart (WordNum : INTEGER) RETURNS INTEGER
	  DECLARE Index, ThisPos, NumFound : INTEGER
	  DECLARE ThisChar : Char
	
	  CONSTANT SPACECHAR = ' '
	
	  Index ← -1
	  Numfound ← 0
	  ThisPos ← 1
	
	  IF WordNum = 1 THEN // if looking for word 1...
	     Index ← 1 // Word 1 always starts at index
	                      // position 1
	  ELSE
	                       // Otherwise start counting spaces...
	     WHILE ThisPos <= LENGTH(FNString) AND Index = -1
	        ThisChar ← MID(FNString, ThisPos, 1)
	        IF ThisChar = SPACECHAR THEN
	           NumFound ← NumFound + 1
	           IF NumFound = WordNum - 1 THEN
	              Index ← ThisPos + 1 // the start of the
	                                  // required word
	           ENDIF
	        ENDIF
	        ThisPos ← ThisPos + 1
	     ENDWHILE
	  ENDIF
	RETURN Index
	
	ENDFUNCTION
	``
	
	1 mark for each of the following:
	
	1 Function heading, including return type and function end
	2 Loop counting spaces until word found or end of `FNString`
	3 extract a character from `FNString` **in a loop**
	4 compare with `SPACECHAR` **and** increment count if equal **in a loop**
	5 compare count with `WordNum - 1` (depending on initialisation value) **in a loop**
	6 if equal then set flag or `Index` to `ThisPos + 1` **in a loop**
	7 Return Index (correctly in all cases / following a reasonable attempt)
	8 Works for special case when looking for word 1
	
	Note: Max 7 marks
		7

7(b)	Marks awarded for any reference to each of the following steps of the algorithm: 1 Mention of variable for use as array index 2 Use of a loop (to check through the array) 3 If word is the same as the current array element then return `FALSE` / set flag 4 If word not already in array, loop to find unused element *(second loop)* 5 Store word in unused element and return `TRUE`, otherwise return `FALSE` VARIATION: 1 Mention of variable for use as array index 2 Use of a loop (to check through the array) 3 Save index of (first) unused element found 4 If word is the same as the current array element then return `FALSE` / set flag 5 If word not already in array **and** unused element available, store word in unused element and return `TRUE` otherwise return `FALSE` **Note: Max 4 marks**	**4**

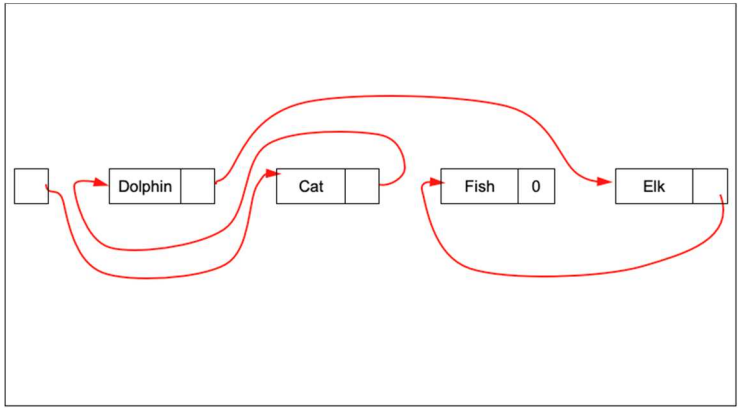
Examiner report missing
9618_s21_qp_23 Question 7

1(a)(i)		**4**

Variable	Example data value	Data type
`Name`	`"Catherine"`	**STRING**
`Index`	`100`	**INTEGER**
`Modified`	`FALSE`	**BOOLEAN**
`Holiday`	`25/12/2020`	**DATE**

One mark per data type

Examiner report missing
9618_s21_qp_22 Question 1

3(a)	Linked list	**1**
3(b)	Start pointer	**1**
3(c)	One mark for each: Name: Null pointer Meaning: There are no further nodes in the list	**2**
3(d)	 One mark for: • Start Pointer pointing to 'Cat' node • Remaining arrows: Cat → Dolphin → Elk → Fish	**2**

Examiner report missing
9618_s21_qp_22 Question 3

5(a)(i)	`DECLARE RNum : ARRAY[1:100] OF INTEGER`
	`DECLARE Index, Count : INTEGER`
	
	`Count ← 0`
	`FOR Index ← 1 TO 100`
	`  RNum[Index] ← INT(RAND(200)) + 1`
	`  IF RNum[Index] >= 66 AND RNum[Index] <= 173 THEN`
	`  Count ← Count + 1`
	`  ENDIF`
	`NEXT Index`
	`OUTPUT Count`
	
	Mark as follows:
	1 Array declaration
	2 Loop for 100 iterations
	3 Array element index 'syntax' (left-hand side of assignment expression) **in a loop**
	4 Use of `RAND()` to generate value in range (and assign to array element) **in a loop**
	5 Check if random number within range **and** if so, increment count **in a loop**
	6 Output of count (following a reasonable attempt) **after the loop**
		6

5(a)(ii)	One mark per bullet / sub-bullet point 1 Initialise the array to a rogue value (to indicate 'unassigned' element) 2 Add a conditional loop to: 3 Generate and store a random number (in the correct range) 4 Check the stored number against values already in the array 5 If the stored number is found then generate another random value 6 Otherwise add it to the array (and exit loop) **Note: Max 3 marks**	**3**

Examiner report missing
9618_s21_qp_22 Question 5

6	`PROCEDURE CountVowels(ThisString : STRING)`
	
	`  DECLARE Index : INTEGER`
	`  DECLARE ThisChar : CHAR`
	
	`  FOR Index ← 1 to 6`
	`  CharCount[Index] ← 0 //initialise elements`
	`  NEXT Index`
	`  Index ← 1`
	
	`  FOR Index ← 1 TO LENGTH(ThisString)`
	`  ThisChar ← LCASE(MID(ThisString, Index, 1))`
	
	`  CASE OF ThisChar`
	`  'a'  : CharCount[1] ← CharCount[1] + 1`
	`  'e'  : CharCount[2] ← CharCount[2] + 1`
	`  'i'  : CharCount[3] ← CharCount[3] + 1`
	`  'o'  : CharCount[4] ← CharCount[4] + 1`
	`  'u'  : CharCount[5] ← CharCount[5] + 1`
	`  'a' TO 'z': CharCount[6] ← CharCount[6] + 1`
	`  ENDCASE`
	`  NEXT Index`
	
	`  FOR Index ← 1 to 6`
	`  OUTPUT CharCount[Index] //output results`
	`  NEXT Index`
	
	`ENDPROCEDURE`
	
	1 mark for each of the following:
	
	1 Procedure heading (with parameter) and ending
	2 Declare local variable for `Index` as loop counter but **not** `CharCount` array
	3 Initialise elements of `CharCount` array to zero
	4 Loop through all characters in `ThisString`
	5 Use of `MID()` to extract single character
	6 Test for each vowel **and** increment associated count
	7 Test for consonants **and** increment associated count
	8 Output the results (supporting text not necessary) **after the loop**
	
	Note: Max 7 if `CharCount` not used to store count
		8

Examiner report missing
9618_s21_qp_22 Question 6

8(a)	`FUNCTION IgnoreWord (ThisWord : STRING) RETURNS BOOLEAN`
	
	`  DECLARE Found : BOOLEAN`
	`  DECLARE Index : INTEGER`
	
	`  Found ← False`
	`  Index ← 1`
	`  ThisWord ← TO_LOWER(ThisWord)`
	
	`  REPEAT`
	`  IF TO_LOWER(IgnoreList[Index]) = ThisWord THEN`
	`  Found ← TRUE`
	`  ENDIF`
	`  Index ← Index + 1`
	`  UNTIL Found = TRUE OR Index > 10`
	
	`  RETURN Found`
	
	`ENDFUNCTION`
	
	1 mark for each of the following:
	
	1 Loop through array elements
	2 Convert both strings to same case
	3 Compare array element with parameter **in a loop**
	4 Set a flag (or similar) if match found (after reasonable attempt at MP3) **in a loop**
	5 Return `TRUE` or `FALSE` in all cases
	
	Note: Max 4 if function declaration incorrect
		5

Examiner report missing
9618_s21_qp_22 Question 8

1(a)		**4**

Example value	Explanation	Variable name	Data type
`"Wong"`	The preferred name of the member joining the football club	**`MemberName`**	**`STRING`**
`FALSE`	A value to indicate whether an existing member of the club lives at the same address	**`FamilyMember`**	**`BOOLEAN`**
`19/02/1983`	When the member joined the football club	**`StartDate`**	**`DATE`**
`1345`	The number of points a member has earned. Members of the club earn points for different activities.	**`Points`**	**`INTEGER`**

One mark for each appropriate variable name plus data type

1(c)	Structure: • Record Justification: • Allows different data types • to be stored under one identifier Mark as follows: One mark for Structure Two marks for justification. **Alternative** • Two (1D) arrays • One of string, one of integer • Where same index links the name with the score	**3**

Examiner report missing
9618_s21_qp_21 Question 1

3(a)(i)	One mark per bullet point: • Data from the arrays is written to the files at the end of the day / before the program is terminated / computer is switched off • Data can then be read from the files at the start of the next day and written to / stored in the arrays • No need to (re-)enter the data manually // only need to enter data once **Note: Max 2 marks**	**2**

3(a)(ii)	• The data is retained when the program is terminated / after the computer is switched off // data is stored permanently // non-volatile storage	**1**

3(a)(iii)	One mark per bullet point: • Data items are combined to form a single string / saved as a single line in the file • Data items are separated by a special character // make each data item a fixed length ALTERNATIVE: • Convert all data items / 'number of people' to strings • Consecutive / each line stores a separate data item	**2**

3(b)	``` PROCEDURE Preview (ThisFile : STRING) DECLARE LineNum : INTEGER DECLARE ThisLine : STRING OPENFILE ThisFile FOR READ IF EOF(ThisFile) THEN OUTPUT "Warning Message" ELSE LineNum ← 1 WHILE LineNum < 6 AND NOT EOF(ThisFile) READFILE Thisfile, ThisLine OUTPUT ThisLine LineNum ← LineNum + 1 ENDWHILE ENDIF CLOSEFILE ThisFile ENDPROCEDURE ``` Marks as follows: 1 Procedure heading (including parameter) and ending 2 File `OPEN` and subsequently `CLOSE` 3 Check if file is empty **and** output a warning message if it is 4 Conditional Loop 5 Output line (including blank lines) and read next line **in a loop**	**5**

Examiner report missing
9618_s21_qp_21 Question 3

5	``` PROCEDURE Sort() DECLARE Temp : INTEGER DECLARE NoSwaps : BOOLEAN DECLARE Boundary, Row, Col : INTEGER Boundary ← 999 REPEAT NoSwaps ← TRUE FOR Row ← 1 TO Boundary IF Result[Row, 2] > Result[Row + 1, 2] THEN FOR Col ← 1 TO 2 Temp ← Result [Row, Col] Result [Row, Col] ← Result [Row + 1, Col] Result [Row + 1, Col] ← Temp NEXT Col NoSwaps ← FALSE ENDIF NEXT J Boundary ← Boundary - 1 UNTIL NoSwaps = TRUE ENDPROCEDURE ``` Mark as follows: 1 Outer loop 2 Inner loop 3 Correct comparison **in a loop** 4 Correct swap of col1 array elements **in a loop** 5 Correct swap of col2 array elements **in a loop** (via loop or separate statements) 6 `'NoSwap'` mechanism: Conditional outer loop including flag reset 7 `'NoSwap'` mechanism: Set flag in inner loop to indicate swap 8 Reducing Boundary **in the <u>outer</u> loop**	**8**

Examiner report missing
9618_s21_qp_21 Question 5

6(a)	One mark per point: 1 Check for a free node 2 Search for correct insertion point 3 Assign data value B to first node in free list / node pointed to by start pointer of free list 4 Pointer from A will be changed to point to node containing B (instead of C) 5 Pointer from B will be changed to point to node containing C 6 Start pointer in free list moved to point to next free node **Note: max 4 marks**	**4**

6(b)	One mark per point: • An array (1D) to store the data **and** a second array (1D) to store the pointers • An (integer) variable to hold the start pointer **and** an (integer) variable to store the next free pointer ALTERNATIVE: • Define a record type comprising a data element and a pointer **and** declare an array (1D) of this type • An integer variable to hold the start pointer **and** an integer variable to store the next free pointer	**2**

Examiner report missing
9618_s21_qp_21 Question 6

7(a)	
	``
	FUNCTION GetStart (WordNum : INTEGER) RETURNS INTEGER
	  DECLARE Index, ThisPos, NumFound : INTEGER
	  DECLARE ThisChar : Char
	
	  CONSTANT SPACECHAR = ' '
	
	  Index ← -1
	  Numfound ← 0
	  ThisPos ← 1
	
	  IF WordNum = 1 THEN // if looking for word 1...
	     Index ← 1 // Word 1 always starts at index
	                      // position 1
	  ELSE
	                       // Otherwise start counting spaces...
	     WHILE ThisPos <= LENGTH(FNString) AND Index = -1
	        ThisChar ← MID(FNString, ThisPos, 1)
	        IF ThisChar = SPACECHAR THEN
	           NumFound ← NumFound + 1
	           IF NumFound = WordNum - 1 THEN
	              Index ← ThisPos + 1 // the start of the
	                                  // required word
	           ENDIF
	        ENDIF
	        ThisPos ← ThisPos + 1
	     ENDWHILE
	  ENDIF
	RETURN Index
	
	ENDFUNCTION
	``
	
	1 mark for each of the following:
	
	1 Function heading, including return type and function end
	2 Loop counting spaces until word found or end of `FNString`
	3 extract a character from `FNString` **in a loop**
	4 compare with `SPACECHAR` **and** increment count if equal **in a loop**
	5 compare count with `WordNum - 1` (depending on initialisation value) **in a loop**
	6 if equal then set flag or `Index` to `ThisPos + 1` **in a loop**
	7 Return Index (correctly in all cases / following a reasonable attempt)
	8 Works for special case when looking for word 1
	
	Note: Max 7 marks
		7

7(b)	Marks awarded for any reference to each of the following steps of the algorithm: 1 Mention of variable for use as array index 2 Use of a loop (to check through the array) 3 If word is the same as the current array element then return `FALSE` / set flag 4 If word not already in array, loop to find unused element *(second loop)* 5 Store word in unused element and return `TRUE`, otherwise return `FALSE` VARIATION: 1 Mention of variable for use as array index 2 Use of a loop (to check through the array) 3 Save index of (first) unused element found 4 If word is the same as the current array element then return `FALSE` / set flag 5 If word not already in array **and** unused element available, store word in unused element and return `TRUE` otherwise return `FALSE` **Note: Max 4 marks**	**4**

Examiner report missing
9618_s21_qp_21 Question 7